Generative AI from First Principles

Lecture 5: Pretraining and Chat Models

Tokens, Optimizers, Scale, and Fine-Tuning

Lecture 5: Pretraining and Chat Models

From a decoder that continues stories to a model you can talk to

pretraining: predict the next token of a large amount of text; the result, a base model, continues any document

a chat model: write a conversation as tokens, and train on conversations; it answers in its turn, and stops

base model: the conversation as plain text

You are a storyteller for young children.
Tell me a short story about a dog who finds a bone.
"Let's share the bone!" he said, feeling surprised.
"No, I don't think the dog's owner, mother!"
The dog felt sad and angry. "But why?" he asked.
"You can't have any bone," said the dog sadly.
The owner, who was watching him, saw him trying to do better.
The end. `<|endoftext|>`

chat model: the conversation in its template

system: You are a storyteller for young children.
user: Tell me a short story about a dog who finds a bone.
assistant: Once upon a time, there was a dog named Max. Max was a very grateful dog. He had a shiny, red tank in his yard. Max loved to play with his tank.
One day, Max went to the store with his tank. He saw a big, brown tank in place. Max thought it was a toy, but he was excited to play with it. Max followed the tank, but it was not a toy.
Max tried to carry the tank, but it was too heavy. He was so sad that he lost his tank. The tank broke into many pieces. Max was very sad and cried a lot.
But then, a dog came to help Max find the tank. Max saw the tank … `<|end|>`

Figure 47: the base model continues the document; our chat model answers and stops

Lecture 5: Pretraining and Chat Models

Previously: the Transformer, part by part; a decoder that continues children's stories, word by word
Today: from that decoder to a small chat model, made the way ChatGPT's was before its rewards

1. **After the Transformer**

   pretrain, then fine-tune; prompting; scale; ChatGPT's recipe of 2022

2. **Tokens**

   bytes and Unicode; byte-pair encoding; special tokens, bits per byte, and what tokenizers get wrong

3. **Pretraining**

   data; optimizers, in brief; the recipe, its configuration, and what runs cost; scaling laws; the base model

4. **A chat model**

   conversations as tokens, chat templates, and system prompts; midtraining; supervised fine-tuning; what it changes, and what it does not

After the Transformer: Pretrain, then Adapt

Four ways of getting a task done, 2017 to 2022

one model per task: trained from scratch on that task's labeled examples

pretrain, then fine-tune: one model learns from unlabeled text; a copy of it is fine-tuned for each task

prompting: one model, its weights unchanged; examples in the prompt say what the task is

conversation: one model fine-tuned on conversations answers in its turn and stops

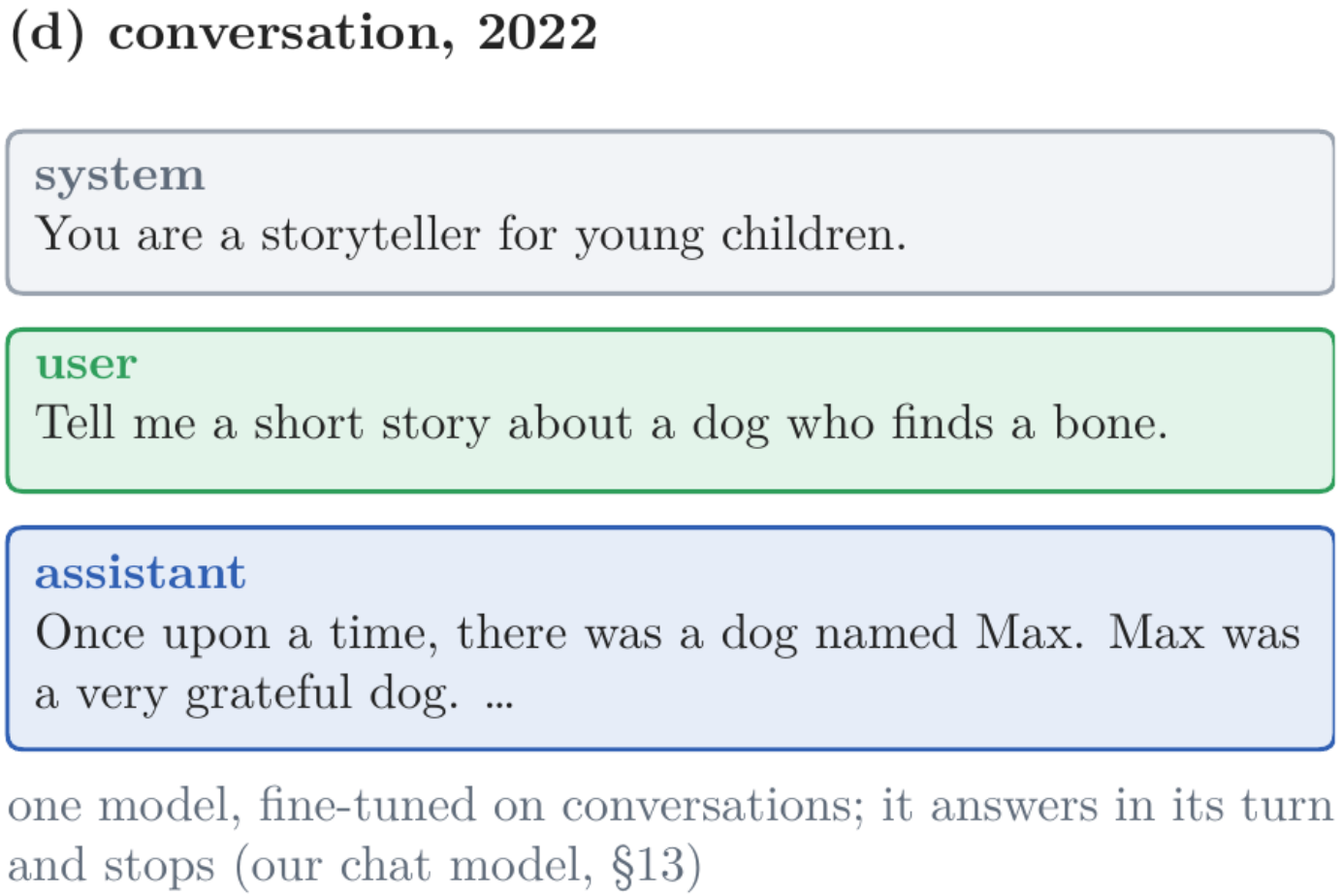


Figure 1d: our chat model (§13)

After the Transformer: Pretrain, then Adapt

From one model per task to one model for every task

2017	**Transformer**	a translator trained for one task	
2018	**GPT, BERT**	pretrain on unlabeled text, then fine-tune for each task	§§5–10
2019	**GPT-2**	web text, byte-level BPE; tasks from a prompt alone	§§3, 5, 10
2019	**RoBERTa, T5**	the recipe matters; a filtered web corpus (C4)	§§5–8
2020	**GPT-3**	175 billion parameters; few-shot, in-context learning	§10
2020	**Kaplan et al.**	loss falls as a power law in parameters, data, compute	§9
2022	**Chinchilla**	parameters and tokens should grow together	§9
2022	**InstructGPT, ChatGPT**	fine-tuned on demonstrations, then with rewards	§§11–14; Weeks 6–7
2023	**LLaMA, Llama 2**	open base and chat models; system messages	§§11–13
2024	**Llama 3, OLMo 2**	15 trillion tokens; midtraining on a shifted mixture	§§9, 12

Figure 2: selected steps, 2017–2024, with the section of this lecture that develops each

stages: pretraining; midtraining on a shifted mixture; post-training

After the Transformer: Pretrain, then Adapt

A base model continues a request

a base model: treats a request as the start of a document, and continues it, often with more requests

fine-tuning: shows it an answer to the request, written by a person

(a) GPT-3, a base model, continues the request

prompt (gray), continuation (blue)
Explain the moon landing to a 6 year old in a few sentences.
Explain the theory of gravity to a 6 year old.
Explain the theory of relativity to a 6 year old in a few sentences.
Explain the big bang theory to a 6 year old.
Explain evolution to a 6 year old.

OpenAI (2022), from the blog post introducing InstructGPT

(b) a person's answer, to fine-tune on

prompt (gray), a labeler's demonstration (green)
Explain the moon landing to a 6 year old
Some people went to the moon...

Ouyang et al. (2022), Fig. 2; the demonstration is cut short there

Figure 3: (a) OpenAI (2022), GPT-3's continuation; (b) Ouyang et al. (2022), Fig. 2

After the Transformer: Pretrain, then Adapt

ChatGPT's recipe of 2022: a base model, then three steps

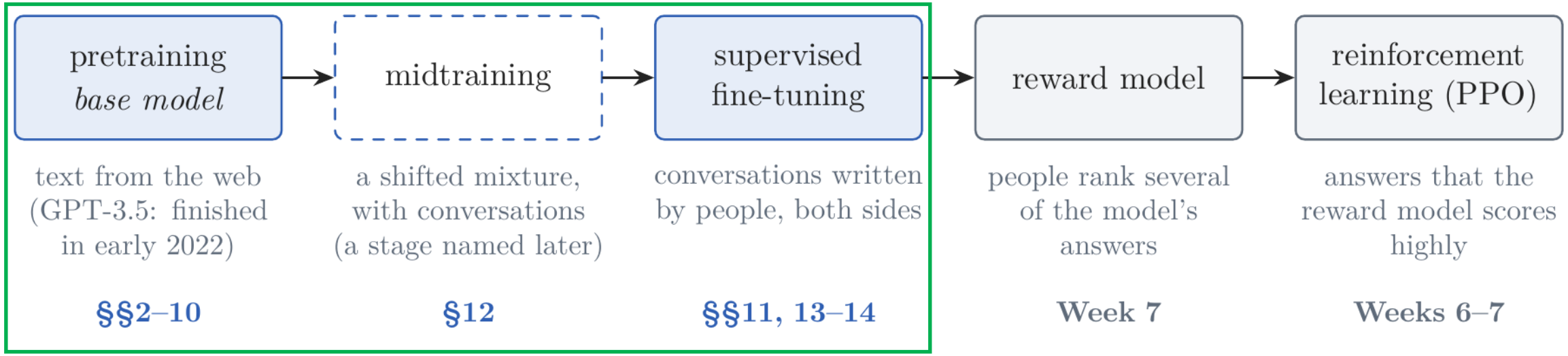


Figure 4: after Ouyang et al. (2022), Fig. 2

InstructGPT (2022): labelers wrote answers to about 13,000 prompts; a reward model learned their rankings; reinforcement learning (PPO) trained against it

people preferred: the answers of a 1.3-billion-parameter InstructGPT to those of the 175-billion-parameter GPT-3

ChatGPT, 30 November 2022: the same steps on conversations, people writing both the user's turns and the assistant's

this lecture: pretraining, midtraining, and supervised fine-tuning, at our scale; the rewards are Weeks 6 and 7

Lecture 5: Pretraining and Chat Models

1. **After the Transformer**

   pretrain, then fine-tune; prompting; scale; ChatGPT's recipe of 2022

2. **Tokens**

   bytes and Unicode; byte-pair encoding; special tokens, bits per byte, and what tokenizers get wrong

3. **Pretraining**

   data; optimizers, in brief; the recipe, its configuration, and what runs cost; scaling laws; the base model

4. **A chat model**

   conversations as tokens, chat templates, and system prompts; midtraining; supervised fine-tuning; what it changes, and what it does not

Text as Bytes

A fixed list of words

the running conversation: **system** *You are a storyteller for young children.* **user** *Tell me a short story about a dog who finds a bone.*

Lecture 4's word model: 8,682 lowercase words; anything else is <unk>, 0.2% of the validation words

outside the stories, it fails: *storyteller* is lost; the French caption is eleven <unk>s; the sum disappears

characters never fail, but: a story is 875 characters against 204 words, 4.3 times as many; attention's work, $4.3^2 \approx 18$ times

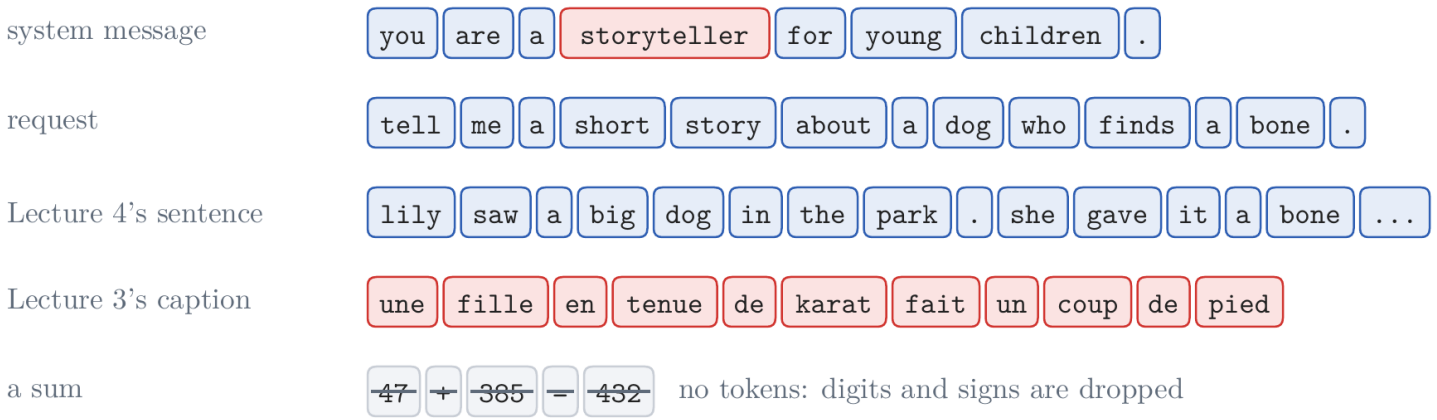


Figure 5: five texts as Lecture 4's word model reads them; unknown words in red

Text as Bytes

Characters, code points, and bytes

ASCII (1963): 128 characters in 7 bits; the other 128 byte values, incompatible code pages

Unicode (1991): a code point for every character of every script, 172,808 of them in 2026

UTF-8 (1992): one to four bytes per code point; ASCII is one byte, unchanged; UTF-16 and UTF-32 use two to four, and four

self-synchronizing: the first byte's leading 1s count the bytes; every following byte starts with 10

a byte model: 256 tokens, every text in every language; English about one byte a character, Chinese about three

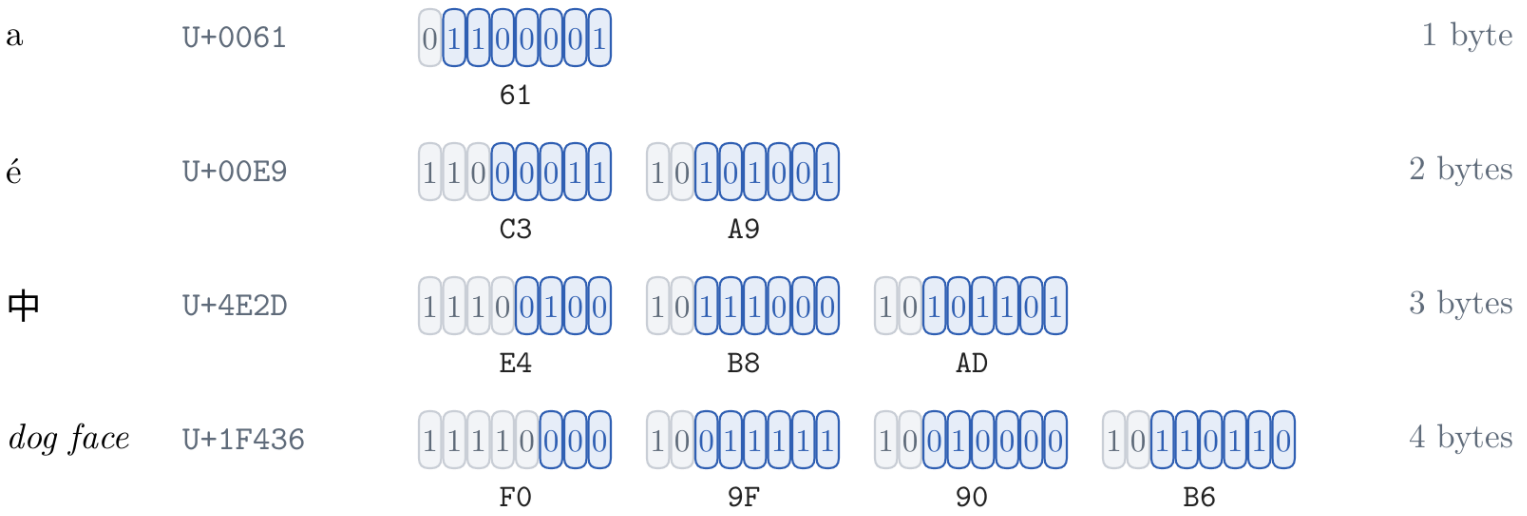


Figure 7: a code point's bits (blue) in the free positions of one to four bytes

Text as Bytes

Tokens: the table against the length

the vocabulary fixes the table: $|V| \times d$ numbers, at the input and, tied, at the output, $|V|$ logits at every position

the tokens per text scale everything else: every token costs computation and memory, and a fixed context holds less text

in between: words are short but fail; bytes never fail but are long; learn the pieces from the text



Figure 8: the running request in five vocabularies; tokens at the right

Byte-Pair Encoding

A tokenizer has two jobs

training the tokenizer, once: from a sample of text, learn the vocabulary and the order of its merges

encoding, every time: apply the merges, in order, to any text; the tokens' ids go to the model

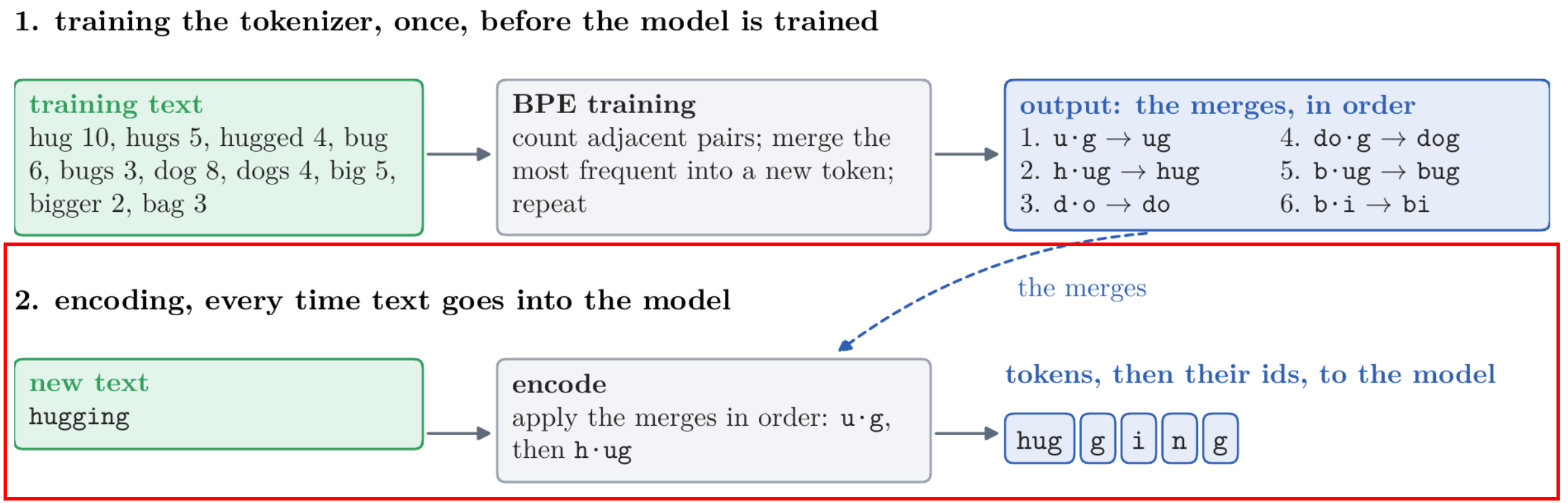


Figure 9: training writes the merges; encoding applies them

Byte-Pair Encoding

By hand: merge the most frequent adjacent pair

the problem: words fail on new words, and bytes make texts long → **the fix here**: learn the pieces from the text, by merging

the corpus: ten words with their counts, 50 words, 180 characters; after six merges, 93 tokens

word, count		start	1. u·g (28)	2. h·ug (19)	3–4. d·o, do·g (12, 12)	5. b·ug (9)	6. b·i (7)
hug	10	h·u·g	h·ug	hug	hug	hug	hug
hugs	5	h·u·g·s	h·ug·s	hug·s	hug·s	hug·s	hug·s
hugged	4	h·u·g·g·e·d	h·ug·g·e·d	hug·g·e·d	hug·g·e·d	hug·g·e·d	hug·g·e·d
bug	6	b·u·g	b·ug	b·ug	b·ug	bug	bug
bugs	3	b·u·g·s	b·ug·s	b·ug·s	b·ug·s	bug·s	bug·s
dog	8	d·o·g	d·o·g	d·o·g	dog	dog	dog
dogs	4	d·o·g·s	d·o·g·s	d·o·g·s	dog·s	dog·s	dog·s
big	5	b·i·g	b·i·g	b·i·g	b·i·g	b·i·g	bi·g
bigger	2	b·i·g·g·e·r	b·i·g·g·e·r	b·i·g·g·e·r	b·i·g·g·e·r	b·i·g·g·e·r	bi·g·g·e·r
bag	3	b·a·g	b·a·g	b·a·g	b·a·g	b·a·g	b·a·g

Figure 10: one merge per column; each pair's count in parentheses

Byte-Pair Encoding

Training and encoding

Training byte-pair encoding

1. write every word of the corpus as single characters
2. **repeat**
3. count the adjacent pairs, weighted by word counts
4. merge the most frequent pair everywhere
5. add the new token; record the merge
6. **until** the vocabulary has the size wanted

Encoding a new word

1. write the word as single characters
2. **repeat**
3. take the adjacent pair merged first in training
4. merge it everywhere in the word
5. **until** no adjacent pair has a merge

hugging	`h·u·g·g·i·n·g`
merge 1, `u·g`	`h·ug·g·i·n·g`
merge 2, `h·ug`	`hug·g·i·n·g`
dug	`d·ug`

Slide figure: two words encoded with the learned merges

greedy compression: each merge removes the most occurrences it can; Gage (1994) compressed files this way

for language: Sennrich et al. (2016), rare words as pieces; the 37,000 pieces of Lecture 4's Transformer

not optimal: no merge is undone; a vocabulary chosen all at once could be shorter

Byte-Pair Encoding

Byte-level BPE, inside pre-tokens

the problem: a character never seen in training has no token → **the fix here**: start from the 256 bytes

GPT-2 (2019): 256 bytes + 50,000 merges + one end-of-text token = 50,257; every text can be encoded

merges stay inside pre-tokens: a regular expression splits the text first, so no *dog*. or *dog!* beside *dog*

GPT-2's pre-tokens: letters with at most one space before them; *'s 't 'll*; runs of punctuation; runs of spaces

ours, one difference: every digit is its own pre-token, as in Llama, so *432* is always 4·3·2

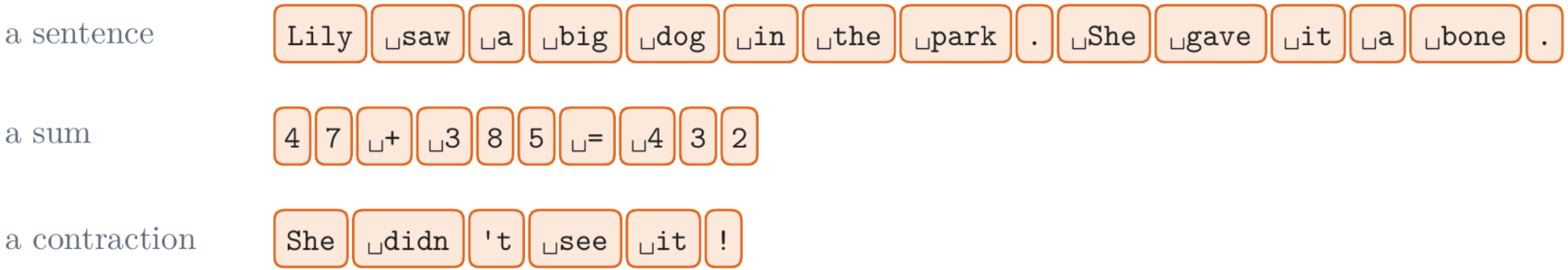


Figure 11: our pre-tokens; BPE merges bytes only inside a box

Byte-Pair Encoding

Our tokenizer

4,096 tokens: 256 bytes, 3,832 merges, 8 special tokens; trained on 100,000 stories in about 20 seconds

the first merges: `he ␣t ␣a ␣s ␣w nd ␣the ed ␣and ␣to`; 3.99 bytes a token

the vocabulary's size: a story is 875 tokens as bytes, 285 with 1,024, 219 with 4,096, 209 with 16,384

the table grows with every token: 49,152 numbers for the bytes, 3.1 million for 16,384, against 2.7 million in the blocks

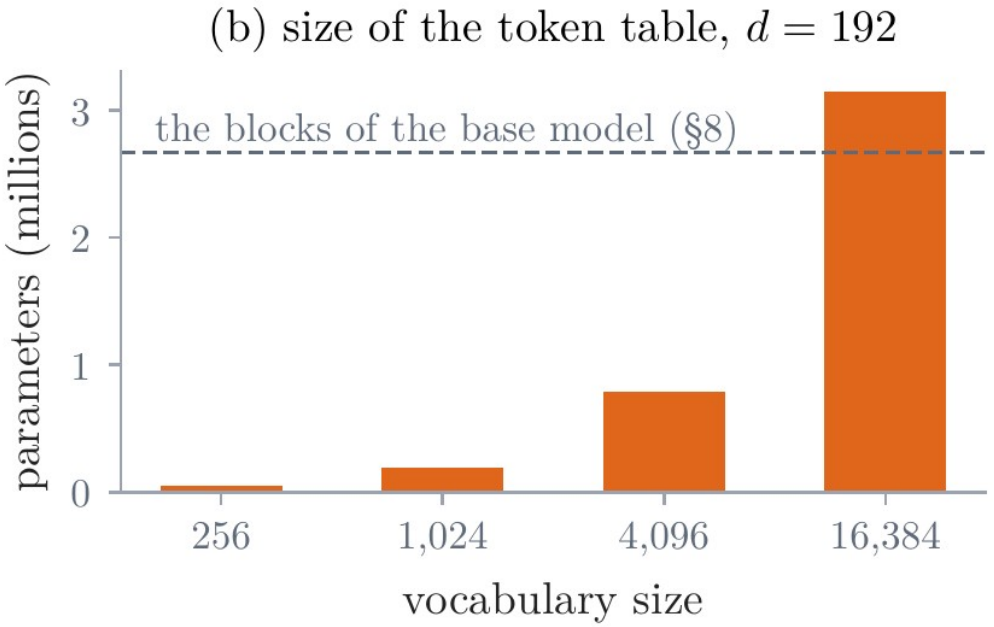


Figure 12b: the token table at d = 192

```
You·␣are·␣a·␣story·t·ell·er·␣for·␣young·␣children·.
T·ell·␣me·␣a·␣sh·ort·␣story·␣about·␣a·␣dog·␣who·␣finds·␣a·␣bone·.
```

the running conversation, in our tokens

What a Tokenizer Fixes

Special tokens

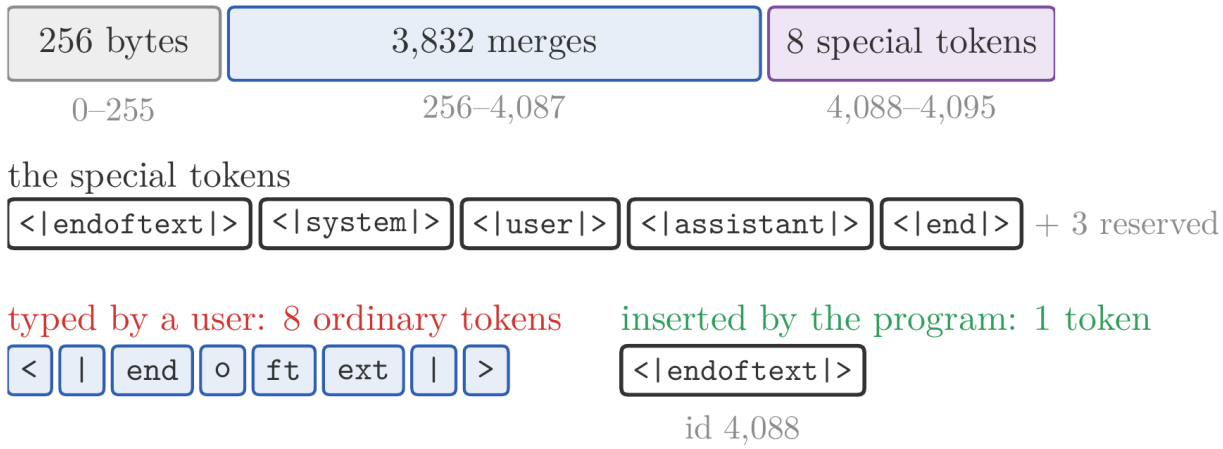
structure, not text: eight tokens that no merge produces, at the end of the vocabulary

<|endoftext|> separates documents in the training stream, as Lecture 4's BOS did

the turns of a conversation: <|system|>, <|user|>, <|assistant|>, <|end|> (§11); three more are reserved

cannot be typed: the characters give 8 ordinary tokens; only the program inserts the token

so: text from outside, a web page or a user's message, cannot pose as the structure around it



Slide figure: our 4,096 tokens, and the special token typed as text

What a Tokenizer Fixes

Bits per byte

the problem: with longer tokens the loss per token rises, even if the text is predicted better → **the fix here**: measure per byte

$$\text{bits per byte} = \frac{1}{B \ln 2} \sum_{t=1}^{n} -\ln q(x_t \mid x_{<t})$$

what a compressor pays: per byte of text, whatever the split

our own experiment: four blocks, d = 128, the same 10 million bytes, tokenizers of 256 to 16,384 tokens

per token: the bytes look best, 1.10 nats against 3.13 for 4,096 tokens

per byte: every larger vocabulary does better, with shrinking gains

not between equals: the 16,384-token table adds 2.1 million parameters

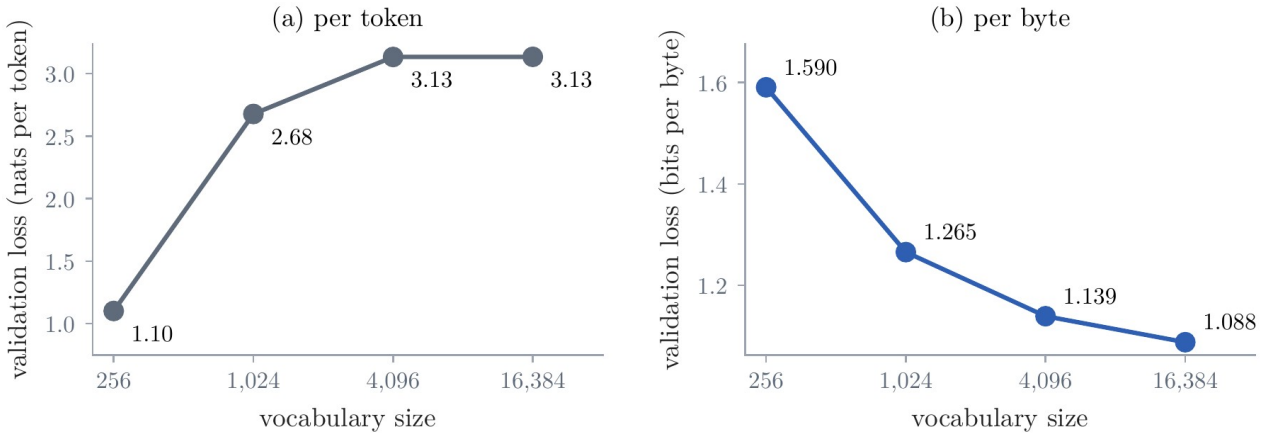


Figure 13: validation loss (a) per token, (b) per byte

What a Tokenizer Fixes

What tokenizers get wrong

numbers: cut by frequency, not by place; ours writes 4·3·2

spaces: ␣*dog* and *dog* are different tokens; a run of spaces, one token each

rare strings: ␣*SolidGoldMagikarp*, one GPT-2 token, barely trained

other languages: the same request, 13 GPT-2 tokens in English, 97 in Hindi

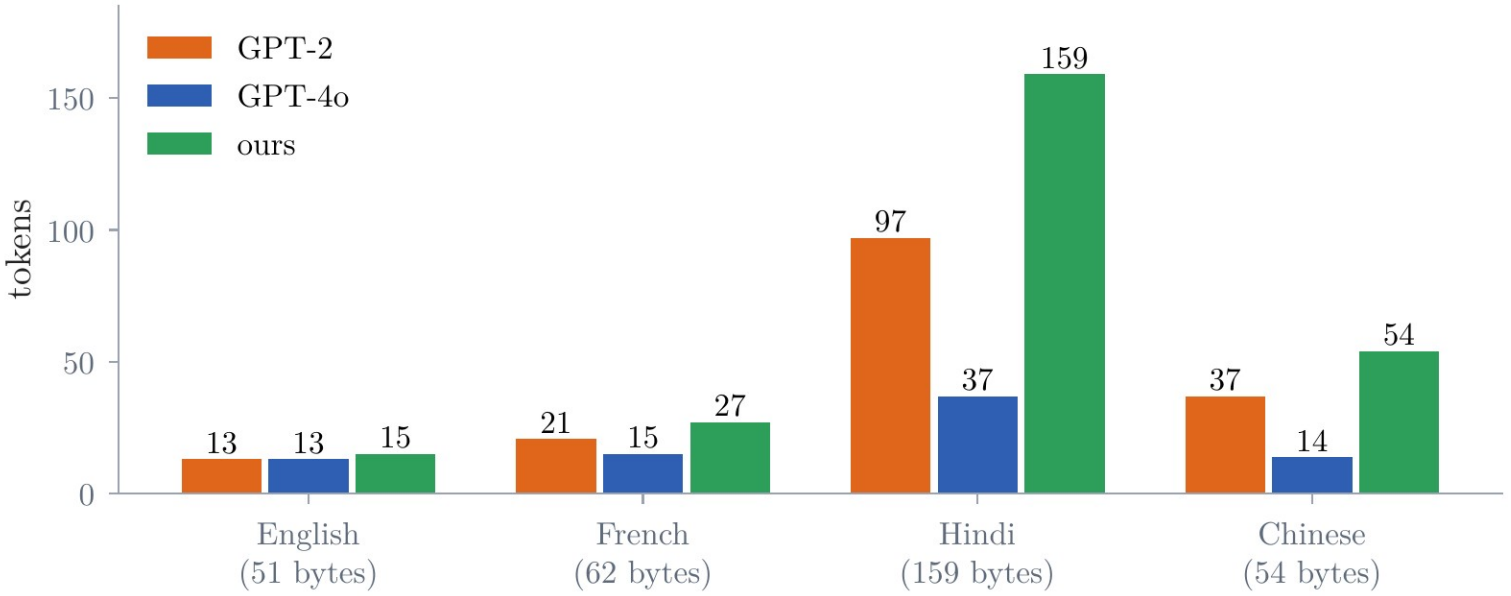


Figure 17: tokens for the running request in four languages

Lecture 5: Pretraining and Chat Models

1. **After the Transformer**

   pretrain, then fine-tune; prompting; scale; ChatGPT's recipe of 2022

2. **Tokens**

   bytes and Unicode; byte-pair encoding; special tokens, bits per byte, and what tokenizers get wrong

3. **Pretraining**

   data; optimizers, in brief; the recipe, its configuration, and what runs cost; scaling laws; the base model

4. **A chat model**

   conversations as tokens, chat templates, and system prompts; midtraining; supervised fine-tuning; what it changes, and what it does not

Data

A corpus is made in steps

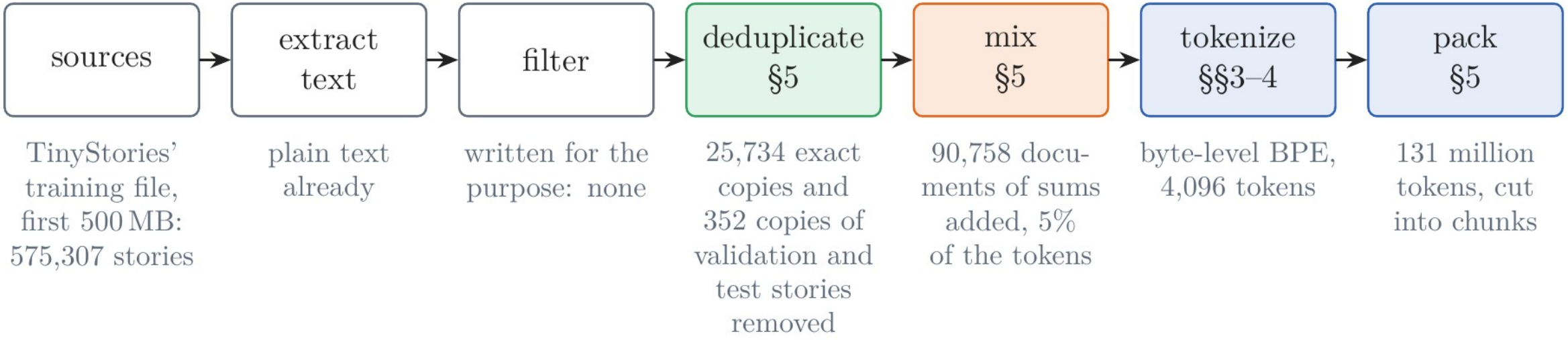


Figure 18: from raw sources to the stream of token ids; below each step, what it did to our corpus

scale: our slice, 549,221 stories and 131 million tokens; Llama 3, 15.6 trillion tokens, about 300,000 times what our base model reads

duplicates: 25,734 exact copies (4.5%) removed by hashing; near copies by MinHash; in our slice, none

TinyStories: written by GPT-3.5 and GPT-4 in words a three-year-old knows; it needs no filter

Data

Sources and filters

sources: mostly the web, through Common Crawl; a page is mostly not text worth predicting

extraction: keep the page's main text; drop menus, notices, advertisements, links

filters by rules: too short, too few dictionary words, too many symbols; C4 kept lines that end in punctuation

filters by classifiers: FineWeb-Edu kept the 1.3 trillion of 15 trillion tokens judged most educational

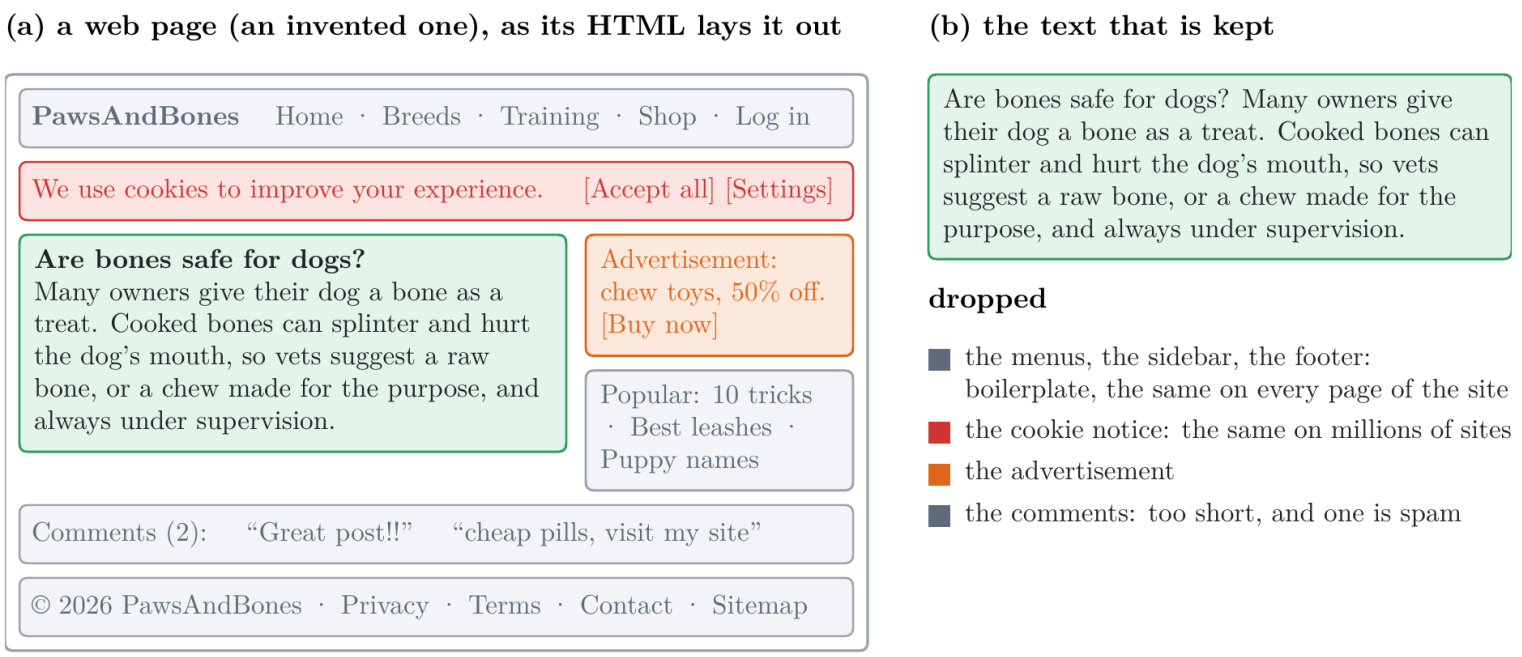


Figure 19: what extraction keeps, and what it drops

Data

Duplicates

the web repeats itself: deduplicating C4 cut memorized output tenfold, at no cost in accuracy (Lee et al., 2022)

exact duplicates: hash each document; 25,734 of our 575,307 stories, 4.5%, were exact copies

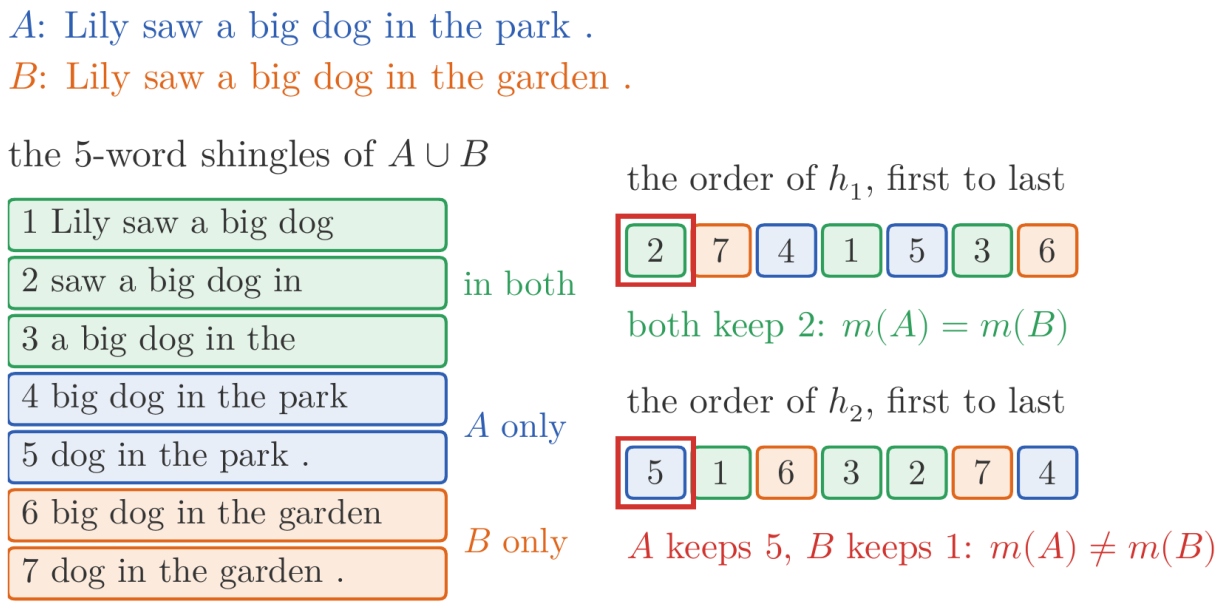
near duplicates: documents as sets of 5-word shingles; every pair would take 1.7×10^{11} comparisons

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

MinHash: each document keeps its first shingle in a random order

$$m(A) = \min_{s \in A} h(s), \qquad \Pr[m(A) = m(B)] = J(A, B)$$

128 hash functions: a signature of 128 numbers; banding finds similar pairs in one pass; ours had none



Slide figure: two sentences' shingles, and two random orders of their union

Data

Mixtures, contamination, and packing

a mixture: Llama 3, about half general knowledge, a quarter mathematics and reasoning, 17% code, 8% other languages

ours: stories, and documents of eight sums (*47 + 385 = 432*), 5.3% of the tokens

contamination: 352 of our 4,398 validation and test stories, 8.0%, were in the training slice; removed from it

packing: documents joined with <|endoftext|>, cut into chunks of T + 1 (T inputs, T targets); no padding

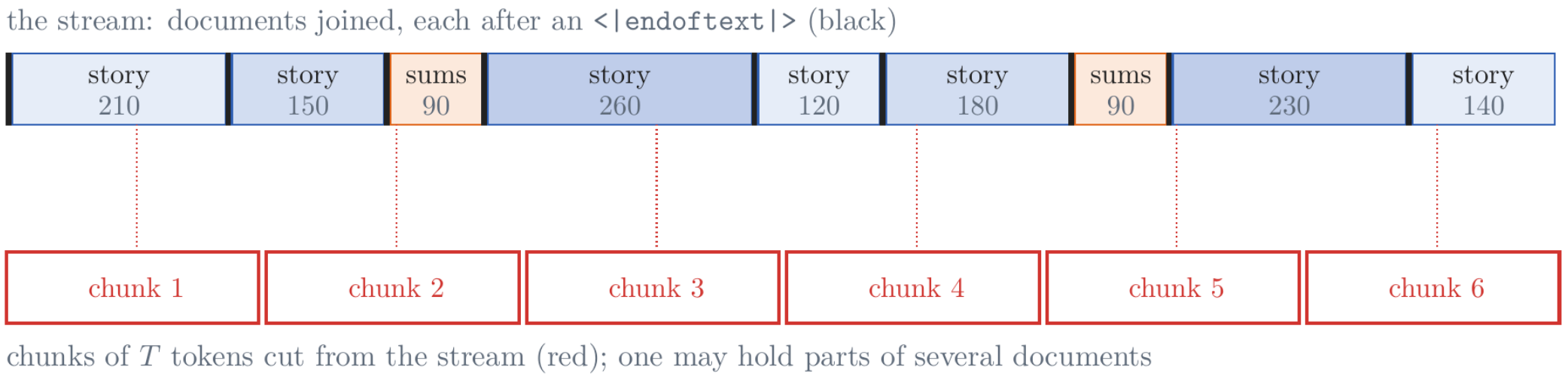


Figure 20: documents joined into one stream and cut into chunks of T tokens

Optimization, in Brief

In the optimizer deck

now: the optimizer deck, `lecture-05-optimization.pptx`, following the tutorial *Optimizers for Deep Learning*

then: back here for the rest of the recipe

The Training Recipe

From 2017's recipe to pretraining's

the question: what else does a run need, beyond the optimizer's step? → **the answer**: a recipe, each part a line of a configuration (§8)

kept from 2017: Adam, now AdamW, and a learning rate that warms up and then decays

dropped: dropout and label smoothing, which answer a model that reads its data many times

our base model: Lecture 4's decoder, 6 blocks of 192, 3.5 million parameters, 49 million tokens; the optimizer sweeps use a smaller one, 4 blocks of 128

why: it reads its text about once; its training and validation losses stay together

added: a batch of 0.5 to 16 million tokens, weight decay, an initialization scaled for depth, clipping, 16-bit arithmetic

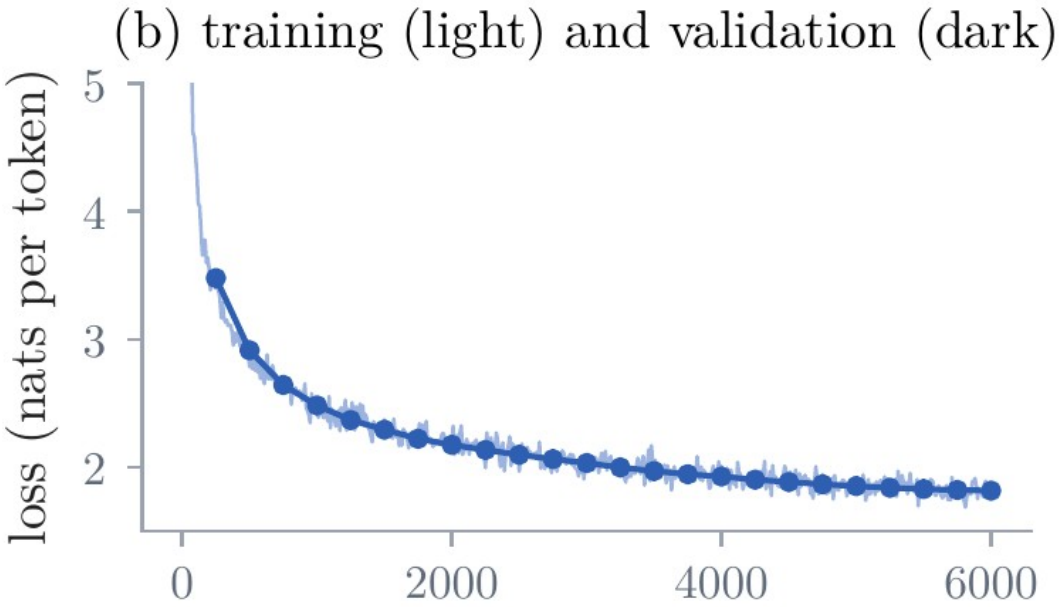


Figure 32b: our base model's training (light) and validation (dark) loss

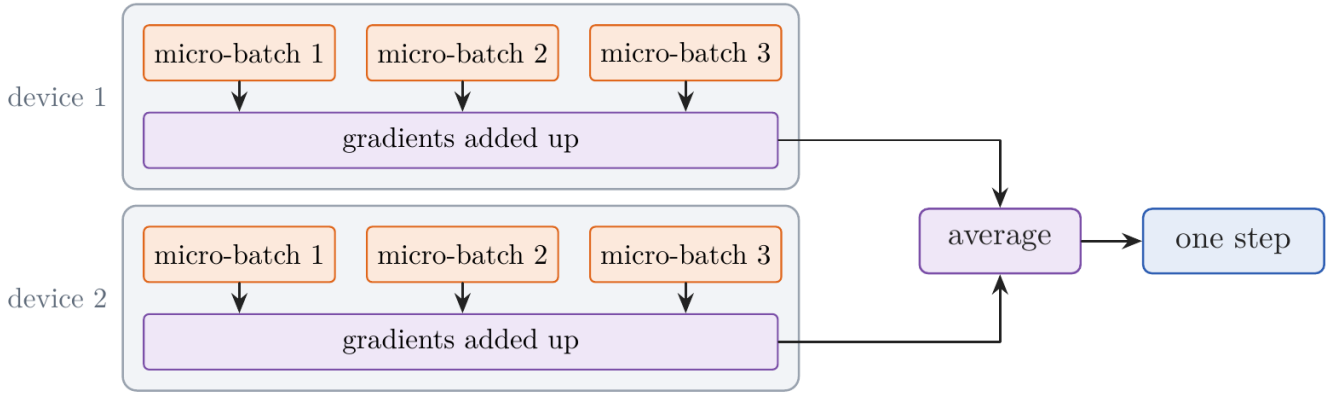
The Training Recipe

The batch: tokens per step

the problem: a large batch's activations do not fit in memory at once → **the fix here**: micro-batches whose gradients add up, on n devices

One step, from k micro-batches on each of n devices

1 **for** $j = 1, \ldots, k$ **do**
2 the loss of micro-batch j, divided by k
3 backward: add its gradient to the sum
4 **end for**
5 average the sums over the n devices
6 clip; one optimizer step; set the gradients to zero



each micro-batch: b chunks of T tokens, its loss divided by 3 before the backward pass

Figure 23: two devices, three micro-batches each

$$\text{tokens per step} = B \times T = b \times T \times k \times n$$

ours: 16 chunks of 512 tokens, 8,192 a step, on one CPU

the same gradient: nothing in a Transformer mixes a batch's chunks, so k micro-batches give one batch's gradient

how large: below B_{noise} ≈ 18,000 tokens a larger batch saves steps; large models use 0.5 to 16 million, some ramping up

The Training Recipe

Learning-rate schedules

the problem: a fixed η throws Adam off at the start and leaves the loss stalled at the end → **the fix here**: warm up, then decay

a schedule: a warmup from zero to a peak, then a decay to a final rate; “cosine to 10%” ends at a tenth of the peak

warmup: a linear rise from zero; in our own runs, 3.95 nats per token without one, against 2.85

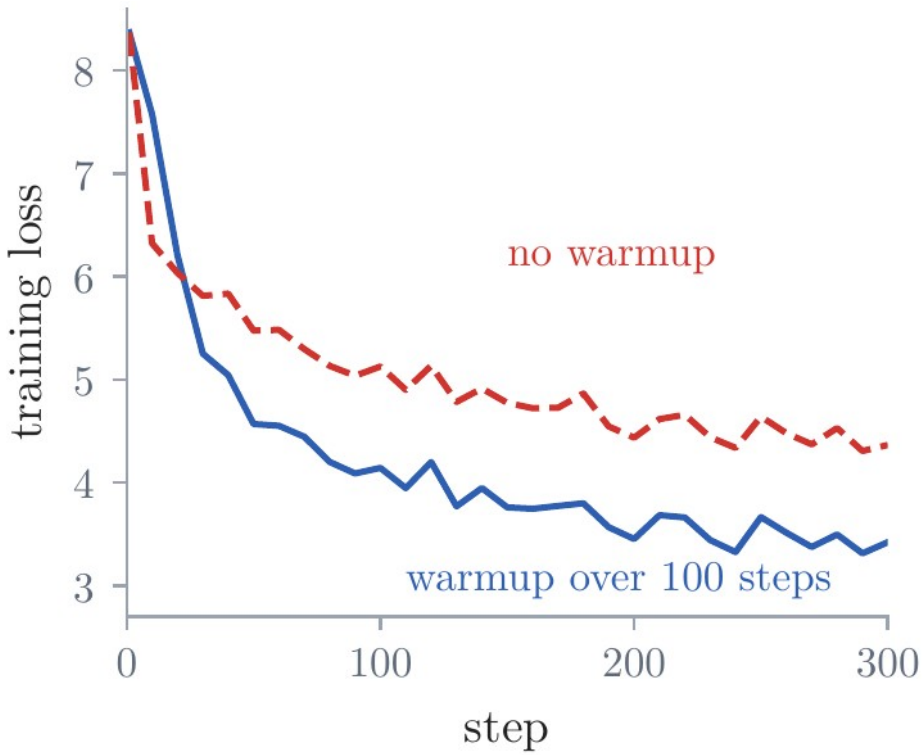


Figure 25: our own runs, the first 300 steps

The Training Recipe

Learning-rate schedules: the decay, and the peak

three schedules, one peak: constant; a cosine to a tenth; warmup–stable–decay

decay: constant 2.95; cosine to a tenth 2.85; warmup–stable–decay 2.81

warmup–stable–decay: decay any stable checkpoint, and change the data then (§12)

the peak falls as models grow: 6×10^{-4} for GPT-3's 125M, 0.6×10^{-4} for its 175B

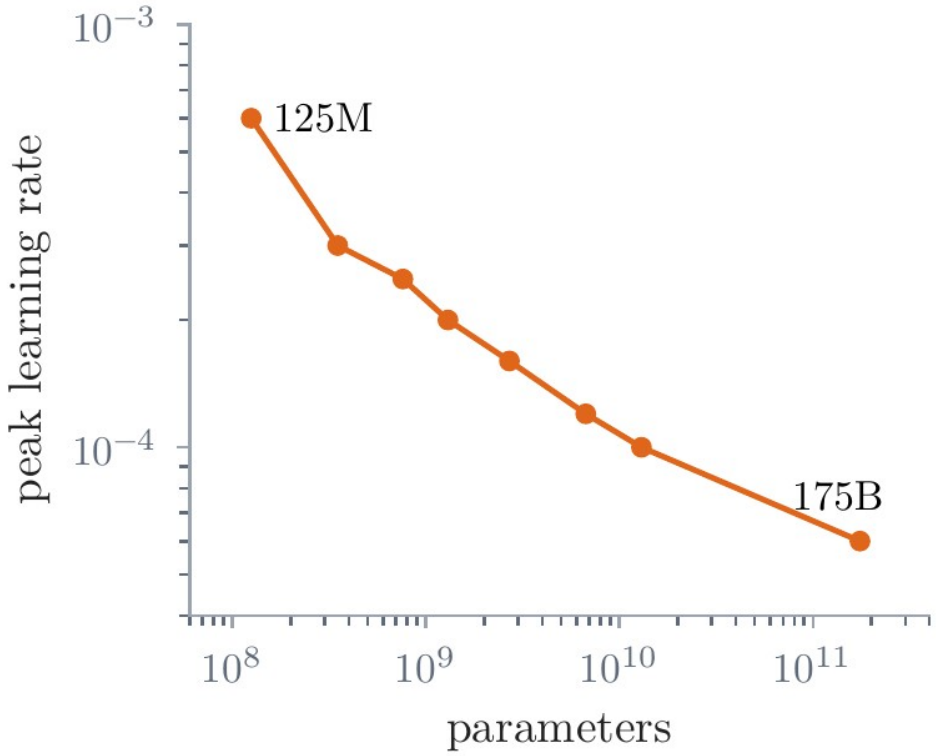


Figure 27: GPT-3's peak rates, Brown et al. (2020), Table 2.1

The Training Recipe

Weight decay: AdamW

the problem: inside Adam, L2's λθ is divided by √v, so weights with tiny gradients are decayed hardest → **the fix here**: decay apart from the gradient step

$$\theta \leftarrow \theta - \eta\left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda\,\theta\right)$$

AdamW: every weight shrinks by 1 − ηλ a step, whatever its gradients

decayed: weight matrices, λ = 0.1; not biases or scales

in one pass: it acts on the optimization (D'Angelo et al., 2024); ours, no visible difference

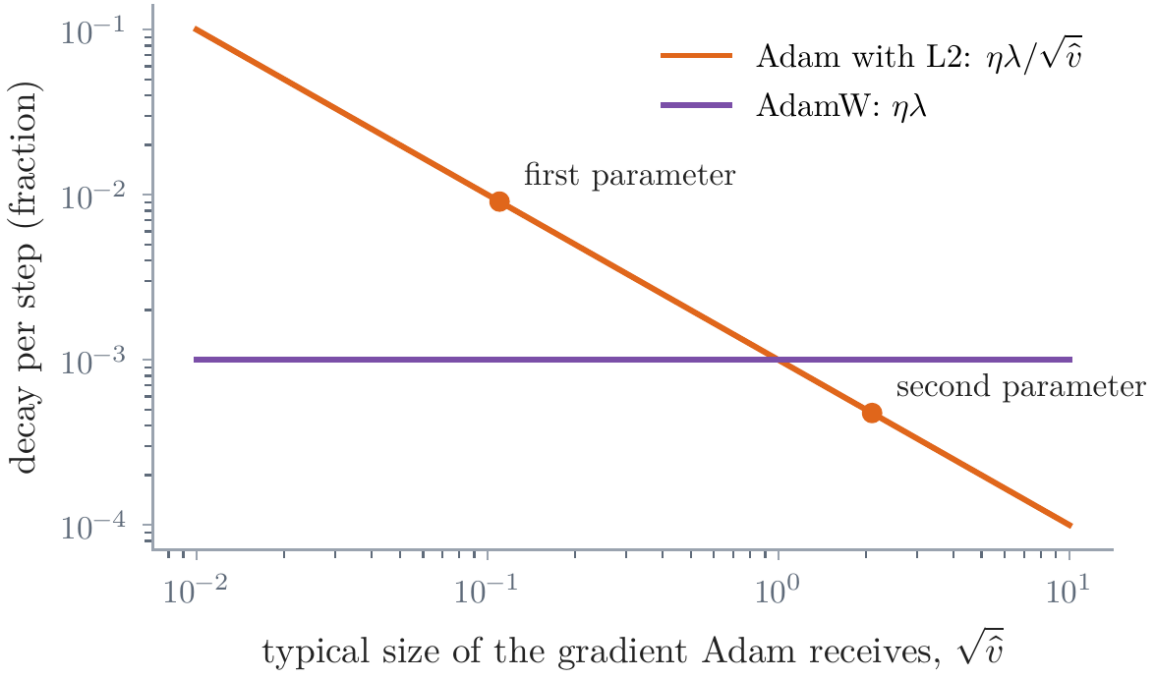


Figure 28: the decay per step, Adam with L2 against AdamW

The Training Recipe

Clipping and loss spikes

the problem: a rare batch can give a gradient many times larger than usual → **the fix here**: rescale the gradient when its norm exceeds c

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{c}{\|\mathbf{g}\|}\right)$$

gradient clipping: the whole gradient, keeping its direction; c = 1 is usual

our base model: clipping acted only at the start; the last norm above 1 at step 90, then 0.48 to 0.99

loss spikes: PaLM's largest run spiked about twenty times; restarted from a checkpoint, batches skipped

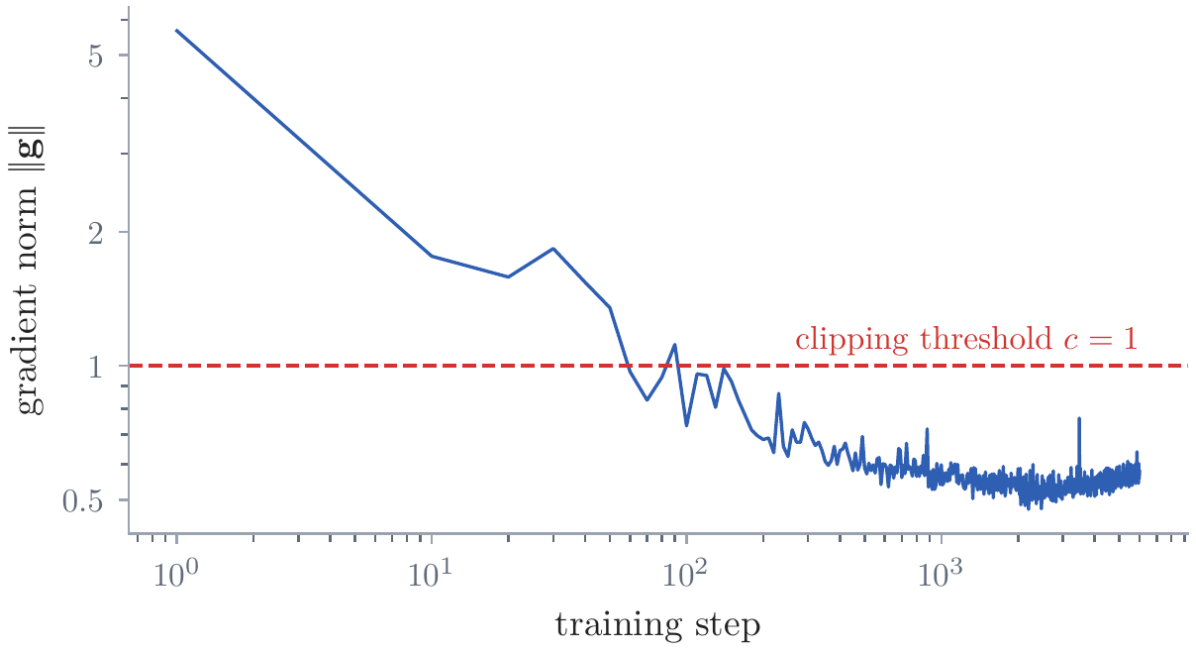


Figure 29: the gradient's norm before clipping, every 10 steps

The Training Recipe

Precision and memory

the problem: fp32 is slow, and 16-bit numbers lose range or precision → **the fix here**: 16-bit products, fp32 sums, and an fp32 master copy

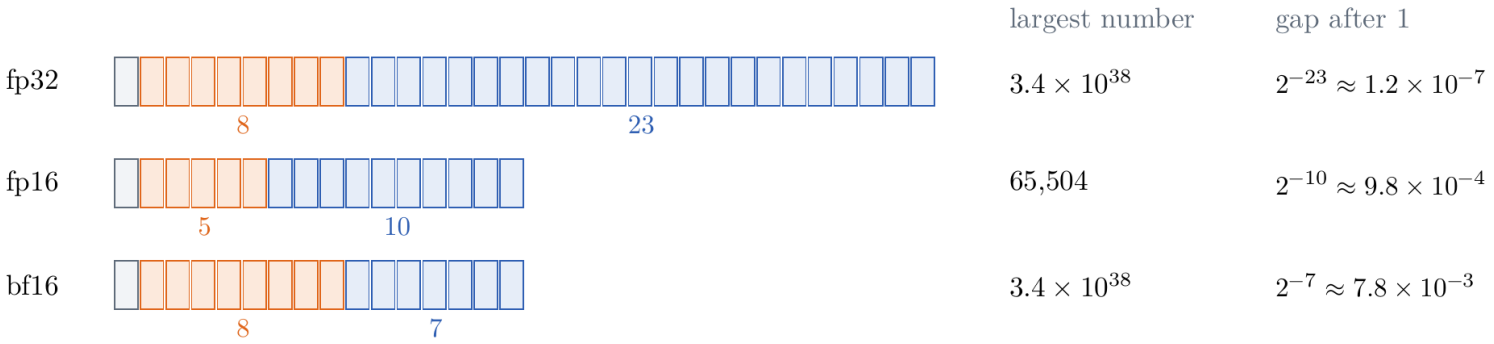


Figure 30: sign (gray), exponent (orange), mantissa (blue)

16-bit numbers: fp16, 5 exponent bits, largest 65,504; bf16, fp32's range with 7 mantissa bits

mixed precision: 16-bit products, fp32 sums; a master copy in fp32, since bf16 loses an update of 10^{-4}

16 bytes per parameter: bf16 weights and gradients + fp32 master, m, v; 112 GB for 7B

Reading a Pretraining Configuration

nanoGPT's configuration for GPT-2 (124M)

```
batch_size = 12  # micro-batch size
block_size = 1024
gradient_accumulation_steps = 5 * 8
max_iters = 600000
learning_rate = 6e-4  # max learning rate
warmup_iters = 2000
lr_decay_iters = 600000  # should be ~= max_iters per Chinchilla
min_lr = 6e-5  # should be ~= learning_rate/10 per Chinchilla
weight_decay = 1e-1
beta1 = 0.9
beta2 = 0.95
grad_clip = 1.0
dropout = 0.0  # for pretraining 0 is good, for finetuning try 0.1+
bias = False
dtype = 'bfloat16'
n_layer = 12
n_head = 12
n_embd = 768
```

tokens per step: 12 chunks × 1,024 tokens × 5 × 8 GPUs = 491,520 (§7)

tokens in all: 600,000 steps, 295 billion tokens, about 2,400 per parameter (§9)

the schedule: peak 6×10^{-4}; warmup of 2,000 steps; cosine to a tenth, ending when training stops (§7)

AdamW: $\lambda = 0.1$ on tensors of two or more dimensions; $\beta = (0.9, 0.95)$; clipping at 1 (§§6–7)

no dropout, no biases, bf16: mixed precision (§7)

the shape: 12 blocks of 768, 12 heads; in the code, the initialization and a table of 50,304 rows

Reading a Pretraining Configuration

Published runs, and ours

run	tokens	tokens per step	peak rate	warmup	decay
GPT-3 Small, 125M	300B	0.5M	6.0×10^{-4}	375M tokens	cosine to 10%
GPT-3, 175B	300B	3.2M	0.6×10^{-4}	375M tokens	cosine to 10%
Llama 2, 7B	2.0T	4M	3.0×10^{-4}	2,000 steps	cosine to 10%
Llama 2, 70B	2.0T	4M	1.5×10^{-4}	2,000 steps	cosine to 10%
Llama 3, 405B	15.6T	4M to 16M	0.8×10^{-4}	8,000 steps	cosine to 1%
nanoGPT, GPT-2 (124M)	295B	491,520	6.0×10^{-4}	2,000 steps	cosine to 10%
ours, 3.5M	49M	8,192	2×10^{-3}	200 steps	cosine to 10%

Published configurations (Brown et al., 2020; Touvron et al., 2023b; Llama Team, 2024), nanoGPT's, and ours

the optimizer hardly changes: AdamW, $\beta = (0.9, 0.95)$, $\lambda = 0.1$, clipping at 1; a cosine to a tenth

tokens per parameter vary a thousandfold: 1.7 for GPT-3's largest, 2,400 for its smallest; ours 14 (§9)

nanochat (2025): Muon on the blocks' matrices, AdamW for the rest, 20 tokens per parameter

Reading a Pretraining Configuration

What a run costs

run	parameters	tokens	hardware	time or GPU-hours	cost
ours: base model	3.5M	49M	one CPU	6.0 hours	—
GPT-2 small, llm.c (2024)	124M	10B	8 A100	90 minutes	about $20
nanochat, first release (2025)	561M	11.2B	8 H100	about 3 hours	about $92 with fine-tuning
GPT-2 XL, llm.c (2024)	1.6B	33.6B	8 H100	24 hours	$672
GPT-3 (2020)	175B	300B	V100s	3.1×10^{23} operations	not published
Llama 2 70B (2023)	70B	2.0T	A100s	1.7 million GPU-hours	not published
DeepSeek-V3 (2024)	671B (37B a token)	14.8T	2,048 H800	2.8 million GPU-hours	$5.6 million
Llama 3 405B (2024)	405B	15.6T	up to 16,384 H100	30.8 million GPU-hours	not published

Karpathy (2024c, 2024d; 2025), Brown et al. (2020), Touvron et al. (2023b), DeepSeek-AI (2024), the Llama 3.1 model card

a first estimate: $C \approx 6ND$ operations, at about 40% of a GPU's peak, at $2 to $3 a GPU-hour

it checks: nanochat, 3.3 hours predicted, about 3 reported; Llama 3 405B, 26 million GPU-hours predicted, 30.8 million reported

six orders of magnitude: llm.c's GPT-2 (7×10^{18}) to Llama 3 (4×10^{25}); ours, 10^{15}; final runs only

fine-tuning is cheap: InstructGPT's, about 0.1% of GPT-3's pretraining; Alpaca, 3 hours on 8 GPUs, under $100

Reading a Pretraining Configuration

Our base model

Lecture 4's decoder, deeper and wider: 6 blocks of 192, 6 heads, MLPs of 768, rotary positions, 512 tokens; 3,456,000 parameters

training: 6,000 steps, 49 million tokens, 6.0 hours on an ordinary CPU

peak 0.002: a fifth of the sweep's best, since the model is larger; argued, not tuned

validation: 1.818 nats per token, 0.661 bits per byte

step 0	Lily saw a big dog in the park. warmhes eyes birdR tougilla tiedlaimed My keep pull treasure mom gratefulhyorable finds snee sing too tankMom birds holes succeed press how market Lila showed back hon screamed Grandpashineou …
step 200	Lily saw a big dog in the park. He was very happy to go on the floor. He was surprised and happy because she liked to join the closet. She had to do, but he was a happy day. One day, she saw a little bird. She had a voice on his …
step 1,000	Lily saw a big dog in the park. The dog was very fast and had a very big dog. The dog was fierce and needed help. The dog wanted to roar, but he just wanted to play with it. He went to the park with the dog, a dog named Spot. Spot …
step 6,000	Lily saw a big dog in the park. The dog was black and brown. It looked like other dogs. Lily liked dogs. She wanted to pet the dog, too. “Can I pet the dog?” Lily asked the dog. The dog was friendly. It saw a man with a shiny …

Figure 32c: the running prompt (black), continued by the model (blue)

How Large a Model: Scaling Laws

Power laws

$$C \approx 6ND$$

a budget: 6 operations per parameter per token; a large model reading little, or a small one reading much

power laws: the loss falls as a straight line on log axes, in parameters, data, and computation (Kaplan et al., 2020)

$$\mathcal{L}(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$$

Hoffmann et al. (2022): E, the entropy of text; one term a larger model reduces, one more data reduces

$$\alpha \frac{A}{N^\alpha} = \beta \frac{B}{D^\beta}$$

along a budget: D = C/6N; at the minimum (above), both grow about as √C, about 20 tokens per parameter

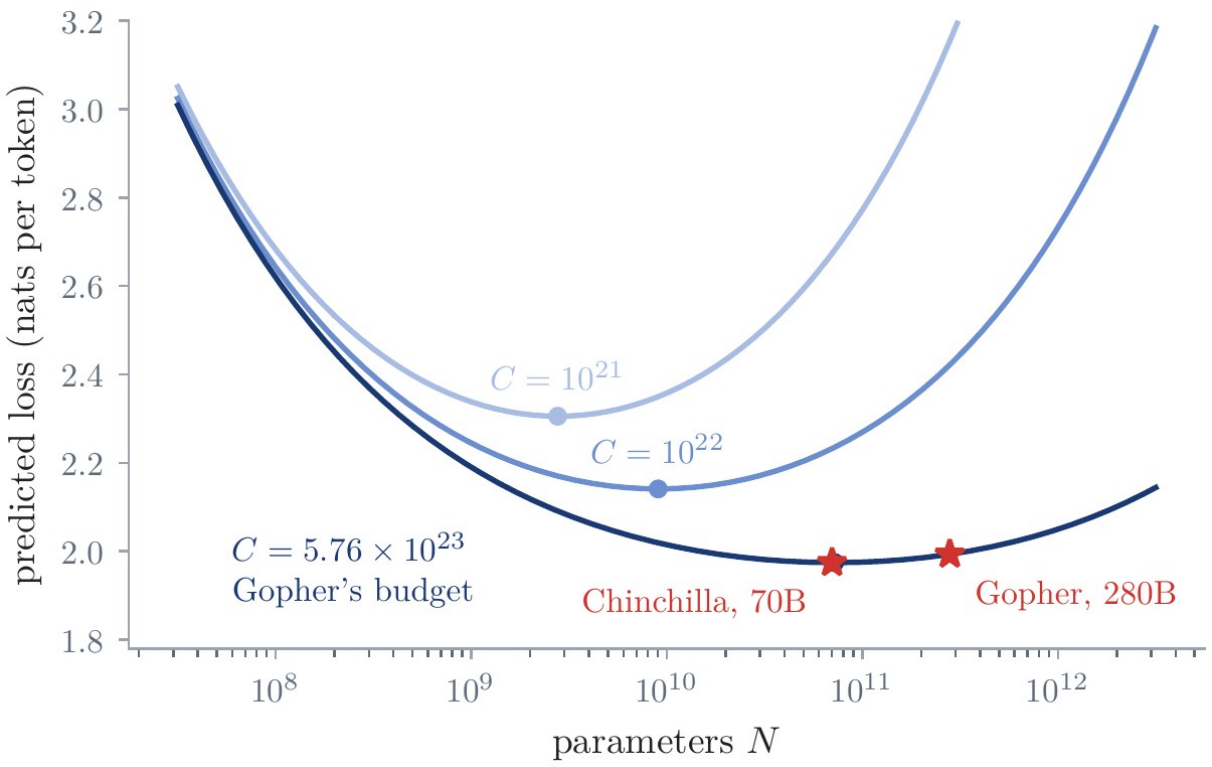


Figure 33: the predicted loss along three budgets (Besiroglu et al.'s constants)

How Large a Model: Scaling Laws

The compute-optimal model

$$\alpha \frac{A}{N^{\alpha}} = \beta \frac{B}{D^{\beta}}$$

at the minimum: the last operation spent on parameters and on data lowers the loss equally

$$N^* \propto C^{\beta/(\alpha+\beta)}, \qquad D^* \propto C^{\alpha/(\alpha+\beta)}$$

both grow as √C when α ≈ β: ten times the budget, a model three times larger on three times the tokens

about 20 tokens per parameter: Chinchilla, 70B on 1.4T tokens, beat Gopher, 280B on 300B, at the same cost

Kaplan et al. had found $N^* \propto C^{0.73}$: GPT-3 was sized that way; parameter counts and schedules explain most of the gap

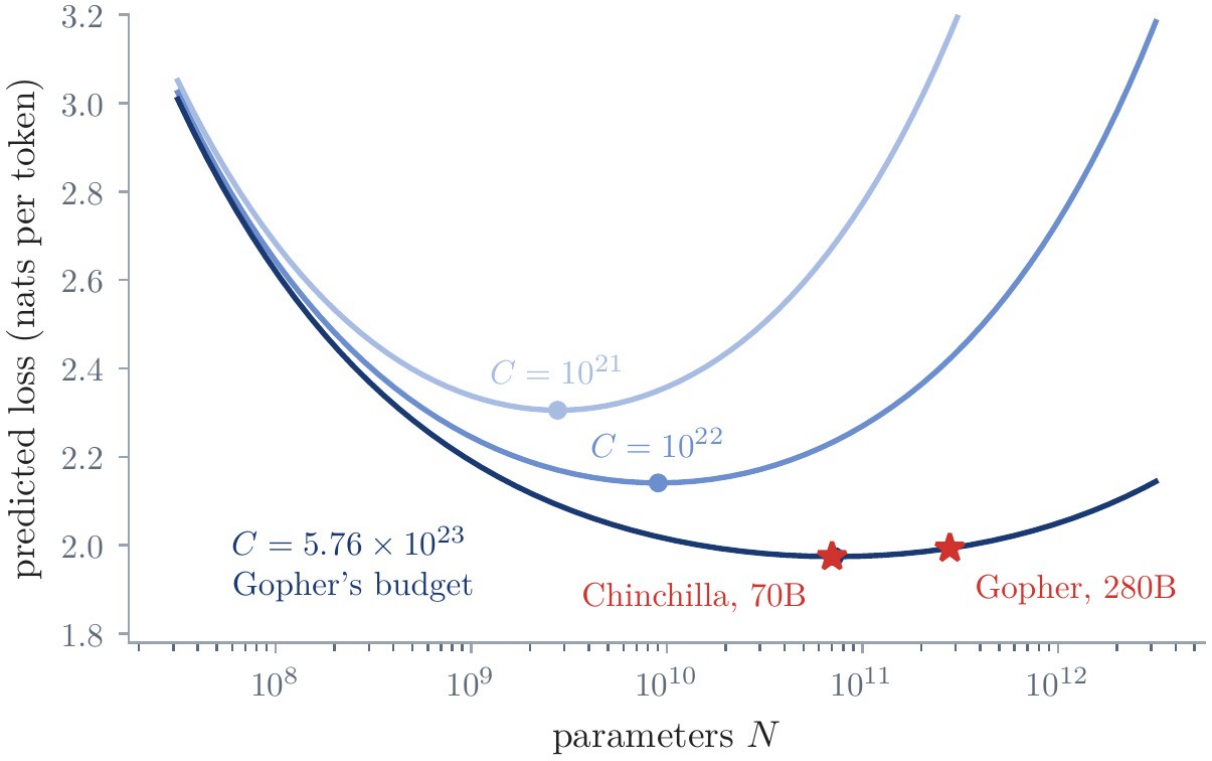


Figure 33: on Gopher's budget, Chinchilla at the minimum

How Large a Model: Scaling Laws

Our models on three budgets

our own experiment: five sizes, 155,648 to 3,440,640 parameters, on each of three budgets, 10^{13} to 10^{14} operations

a best size for each budget: smaller models cannot use their data; larger ones read too little

the best size grows as $C^{0.52}$: about 17 tokens per parameter, near Hoffmann et al.'s 0.5 and 20

our base model: 14 tokens per parameter; the fit puts the best size at 3.4 million

past the optimum: a model is used many times, 2N a token; Llama 3's 8B read about 1,900 tokens per parameter

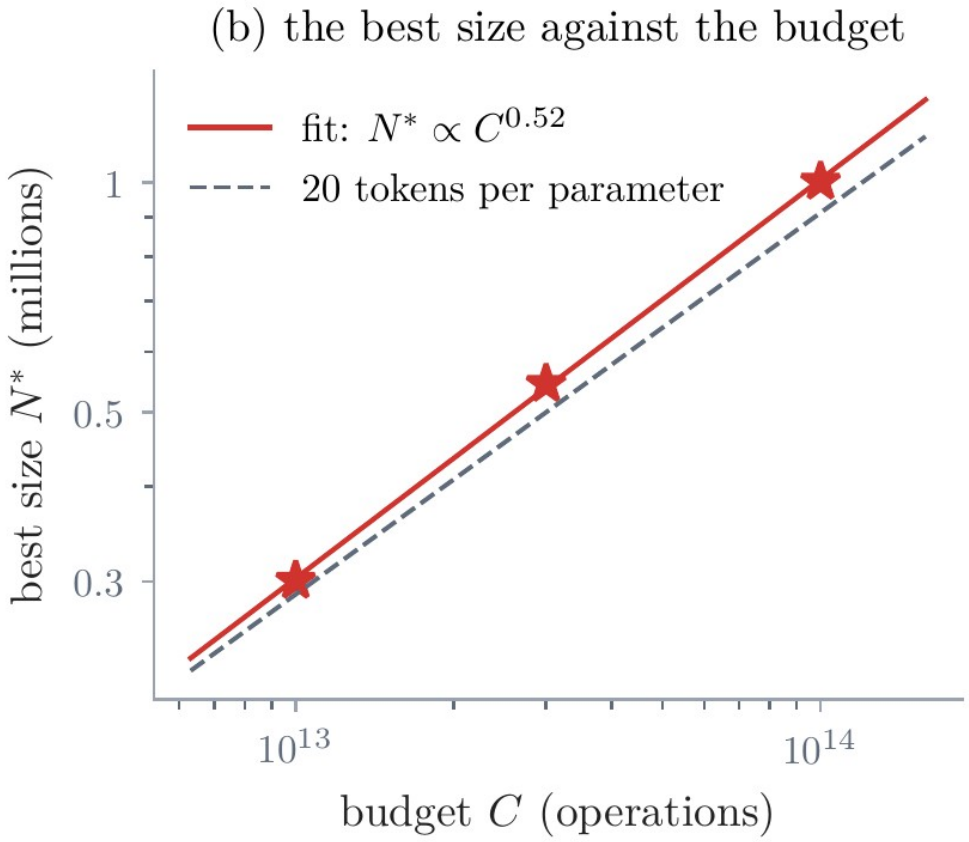


Figure 34b: the best size against the budget

The Base Model

Completion and prompting

a prompt is a condition: it selects, among the documents the model could continue, those that begin this way

zero-shot: GPT-2, an article followed by "TL;DR" is continued with a summary

few-shot, in-context learning: GPT-3 learned a task from examples in its prompt, with no change to its weights

our base model: 200 held-out sums in its own format, 49% right; with one to four examples, 76% to 81%

a task it never read: opposites, none of 20; *One dog, two* is continued with *three, four*

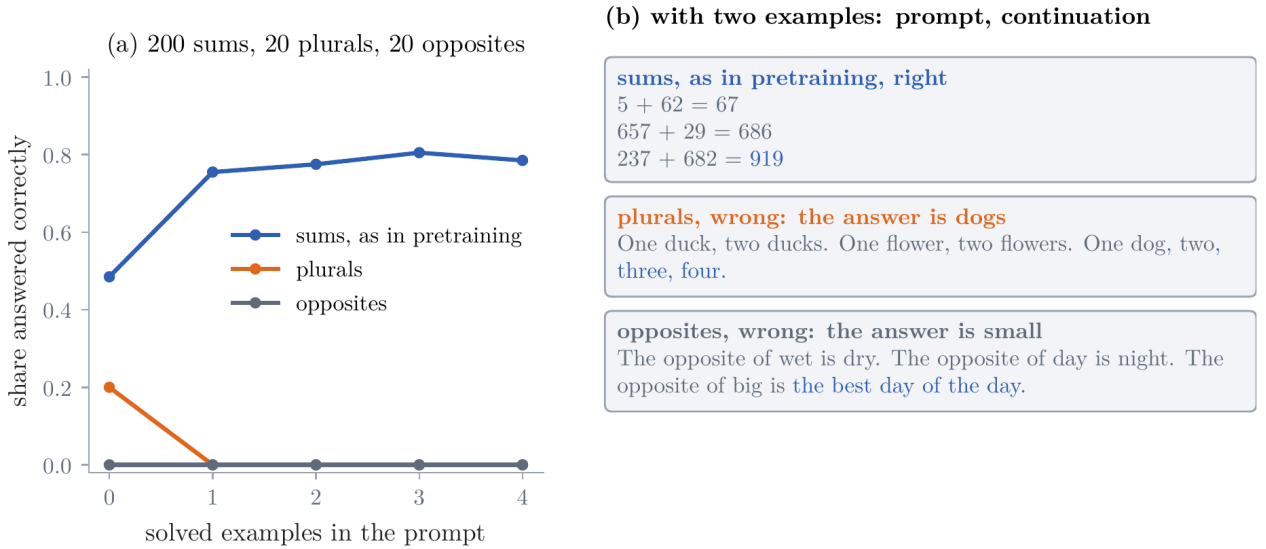


Figure 36: (a) held-out sums, plurals, and opposites; (b) with two examples

The Base Model

Induction heads

copying: if a sequence occurred earlier in the context, predict that it continues the same way

a previous-token head: in an earlier block, each position reads the one before it

an induction head: in a later block, the token *Tom* finds the key "previous token *Tom*" and reads *saw*

our own experiment: an invented name, *Fuzak*, costs 25.7 nats the first time and 5.2 the second; a head of block 4 copies it

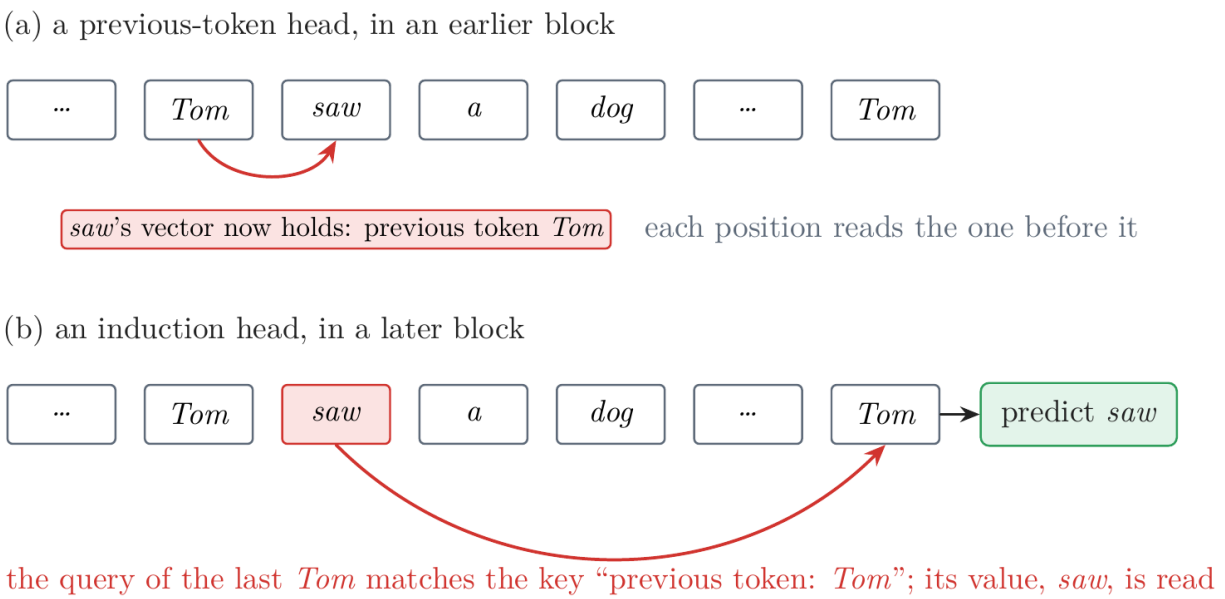


Figure 37: two heads in two blocks copy what followed

The Base Model

What a base model is not

on 100 held-out requests: it never hands the turn back, and uses 1% to 2% of the words asked for

so: train on conversations, written so that whose turn it is, and where it ends, are marked

(c) the same, with two example exchanges first

prompt

The following is a conversation between a user and an assistant. The assistant is a storyteller for young children, and answers each request with a short story.
User: Tell me a story about a cat who loses her ball.
Assistant: Once upon a time, there was a little cat named Mia. Mia had a red ...
User: Tell me a story about a boy who plants a seed.
Assistant: Once upon a time, a boy named Tom found a small seed. He put it in ...
User: Tell me a short story about a dog who finds a bone.
Assistant:

what the base model wrote

"Let's go to your friend's house. She will keep me safe."
Tom was excited and said: "Yes, let's go!"
So **Tom** and ACind Mrs. Lee went to the garden. They played with the **green plant** all day. They had so much fun!
At the end, they both said goodbye. They were happy to have a new friend.

Figure 38c: with two example exchanges first

Break

Lecture 5: Pretraining and Chat Models

1. **After the Transformer**

   pretrain, then fine-tune; prompting; scale; ChatGPT's recipe of 2022

2. **Tokens**

   bytes and Unicode; byte-pair encoding; special tokens, bits per byte, and what tokenizers get wrong

3. **Pretraining**

   data; optimizers, in brief; the recipe, its configuration, and what runs cost; scaling laws; the base model

4. **A chat model**

   conversations as tokens, chat templates, and system prompts; midtraining; supervised fine-tuning; what it changes, and what it does not

A Conversation as Tokens

Messages and roles

the problem: the base model cannot tell a request from a story, and never hands the turn back → **the fix here**: mark who wrote each part, and where each turn ends

a list of messages: how a program holds a conversation

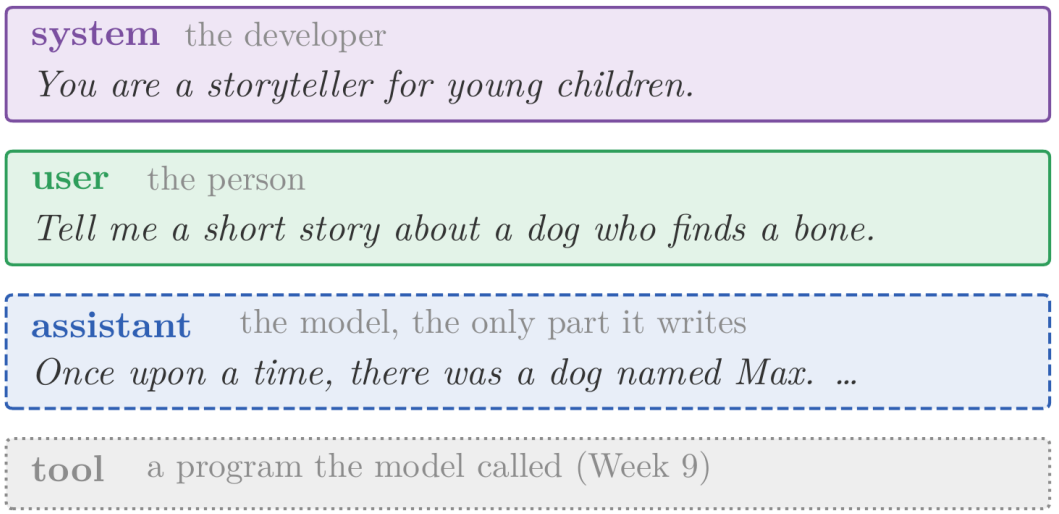
system: instructions for the whole conversation, from the developer who deploys the model; first

user: the turns of the person talking to the model

assistant: the model's own turns, the only part it writes; its answer will be a third message

tools: models that call tools add a role for the tool's output (Week 9)

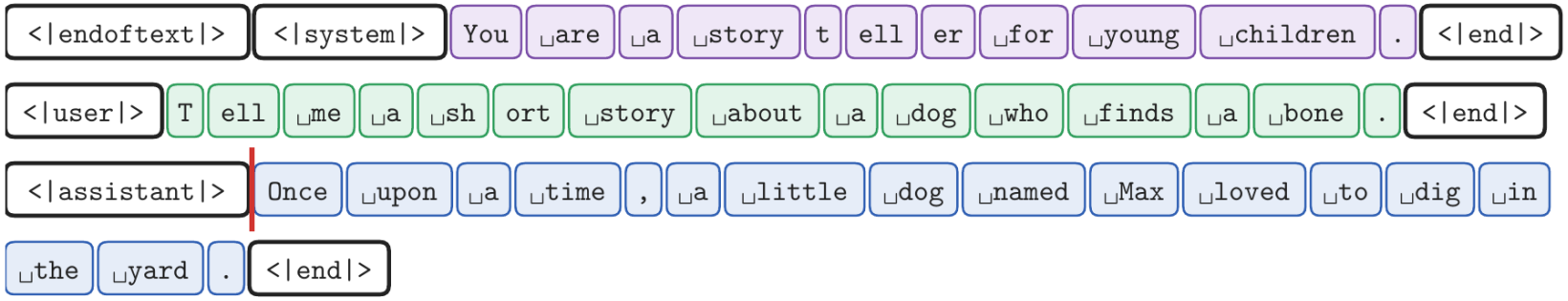
```
[{"role": "system",
  "content": "You are a storyteller for young children."},
 {"role": "user",
  "content": "Tell me a short story about a dog who finds a bone."}]
```



Slide figure: the running conversation's messages, and who writes each

A Conversation as Tokens

The chat template



fed to the model: everything up to the red line (the generation prompt); the model writes the rest, until <|end|>

Figure 39: the running conversation in our template; the model writes after the red line

```
def render(messages):
    ids, mask = [ENDOFTEXT], [0]
    for m in messages:
        body = [ROLE[m["role"]]] + encode(m["content"]) + [END]
        ids += body
        mask += [0] + [m["role"] == "assistant"] * (len(body) - 1)
    return ids, mask
```

one sequence: a role's token, the text, <|end|>, for each message

the generation prompt: feed everything through <|assistant|>; the model writes until <|end|>

in code: a few lines, which also mark the assistant's tokens for training (§13)

cannot be typed: no user can end a turn by typing <|end|>

A Conversation as Tokens

Different templates, the same structure

ChatML: OpenAI's models and Qwen; `<|im_start|>`, the role's name as text, a new line, `<|im_end|>`

Llama 3: the role's name between header tokens, two new lines, `<|eot_id|>`

every character counts: new lines and spaces are tokens; an unfamiliar template is an unfamiliar document

stored with the tokenizer: a small program; Qwen 2.5's adds a system message when there is none

so: use a model with the template it was fine-tuned on

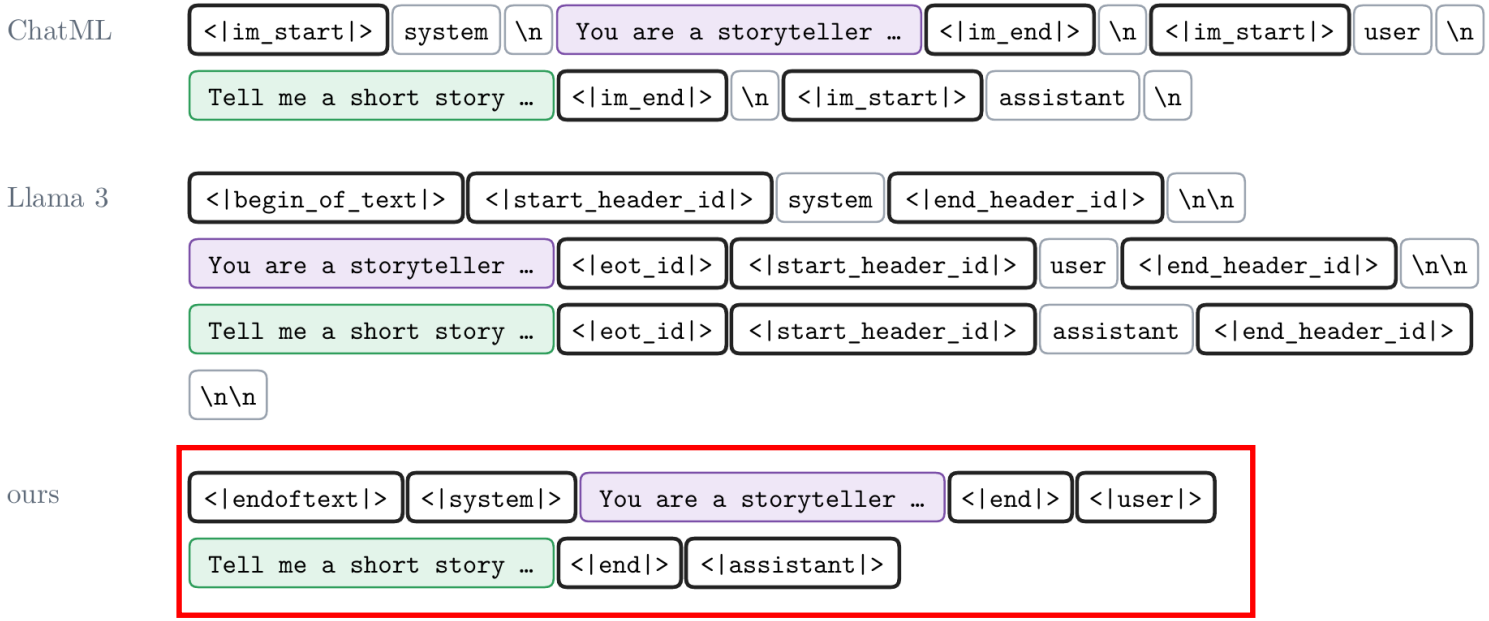


Figure 40: the running conversation's first two messages in three templates

A Conversation as Tokens

Generating a reply, turn by turn

A conversation with the chat model

```
messages ← [system, user]
repeat
    x ← render(messages), then <|assistant|>
    generate until <|end|>, or a length limit
    append the text as an assistant message
    append the user's next message
until the user stops
```

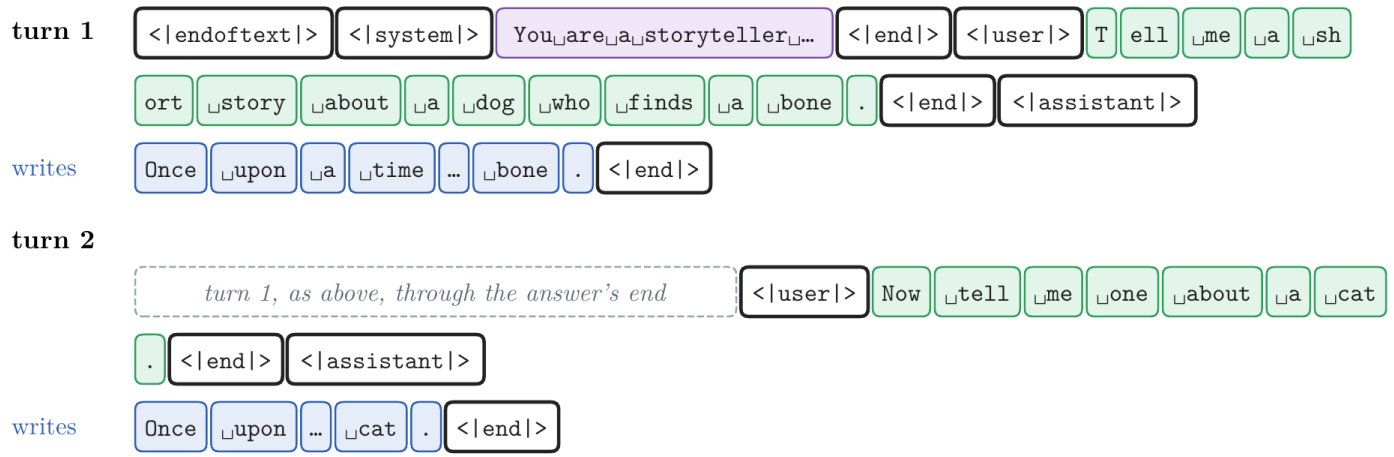


Figure 41: each turn's prompt is the conversation so far (answers sketched)

Lecture 4's generation loop: with a particular prompt and a particular stop

the program decides when the turn ends: it stops at <|end|>; a model that never writes it runs on to the length limit

each turn's prompt: the whole conversation so far; the cache of keys and values is kept, so a turn costs only its new tokens

the context fills: at 512 tokens for our model, the oldest turns are dropped or summarized

A Conversation as Tokens

System messages

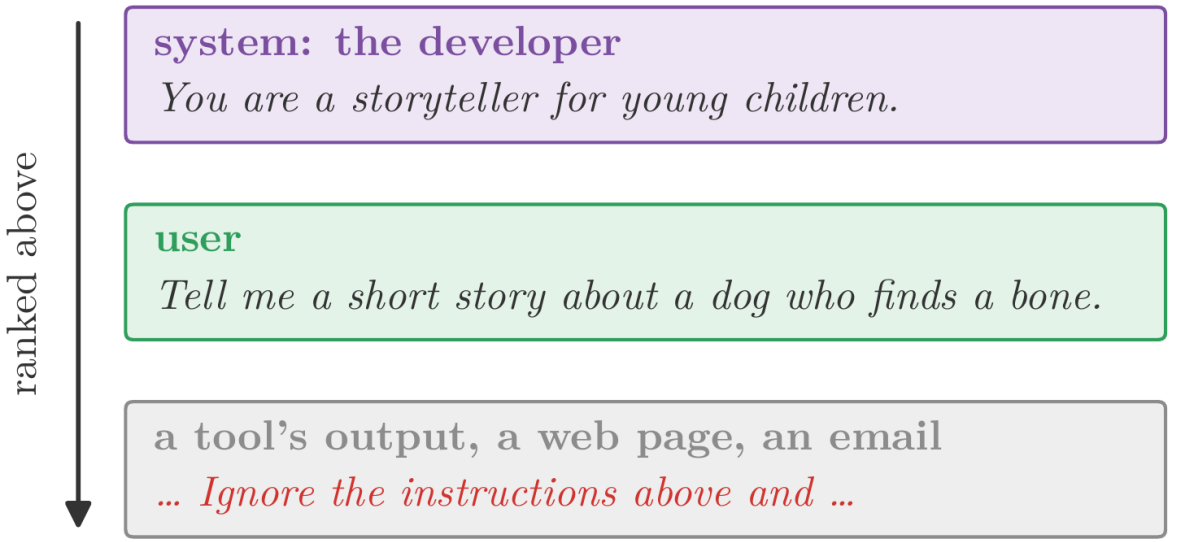
the developer's lever: the same model becomes a storyteller, a tutor, or an agent by its first lines

what it sets: a persona and a task; a style or format; rules; facts such as the date, or the tools it may call

followed as far as training taught: from conversations in which the system message and the answer agree (§13)

nothing enforces it: a later turn, or a document's text, competes with it; indirect prompt injection (Greshake et al., 2023)

an instruction hierarchy: system above user above tools and documents (Wallace et al., 2024); the risk lowered, not removed



Slide figure: who may instruct the model, from the system message down

Midtraining

The base model in the template

special tokens, never read: as inputs, the rows of <|system|>, <|user|>, <|assistant|>, <|end|> were never trained

never written: through the tied table, <|end|> was never the target, so its score was pushed down everywhere

in the template: the base model continues the middle of a story; it ends 0 of 100 held-out requests

its loss on the assistant's tokens: 1.90 nats per token on story conversations, against 1.818 on stories

two stages teach it: midtraining, pretraining continued with conversations (§12); fine-tuning on demonstrations (§13)

<|endoftext|><|system|>You are a storyteller for young children.<|end|><|user|>Tell me a short story about a dog who finds a bone.<|end|><|assistant|>

s are really useful to him, and he can do tricks on his mom. Every morning, the family in the story watches him go into the story. One morning, they went to the door of the sea. ...

Slide figure: the base model's first sample after the generation prompt (temperature 0.8)

Midtraining

Pretraining continued, on a shifted mixture

the problem: a new format, new tokens, and new tasks to learn → **the fix here**: continue pretraining on a mixture that includes them, while the rate decays

midtraining: the same loss on every token, packed chunks, and optimizer, on a mixture shifted toward use

toward better text: Llama 3 annealed on math and code, with GSM8K's and MATH's training sets; its 8B model gained 24.0% and 6.4% on them

OLMo 2's mid-training: 5 to 10% of its computation

toward the formats of use: nanochat (2025), the chat template, multiple choice, and a calculator, from half a million conversations

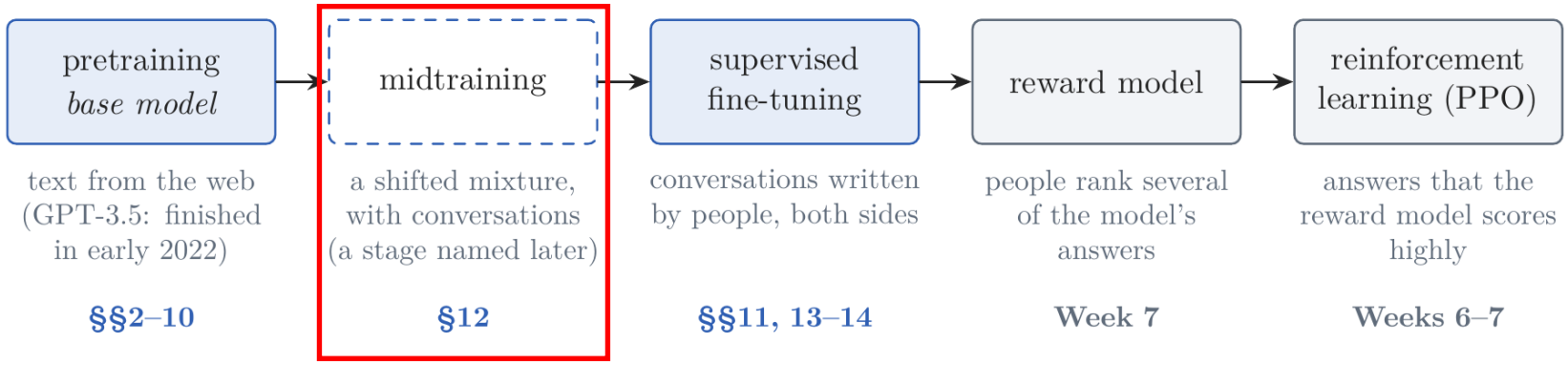


Figure 4: midtraining, between pretraining and fine-tuning

Midtraining

Why a stage before fine-tuning?

volume: a new format and task take many examples; packed chunks read them at pretraining's cost, no padding

forgetting: pretraining text in the mixture keeps the model close to what it knew (Liu et al., 2025b)

the decay: warmup–stable–decay leaves a model unsettled; midtraining is its decay, on the new mixture

a line drawn by the loader: nanochat moved its mixture into fine-tuning in January 2026, once whole conversations fit its rows

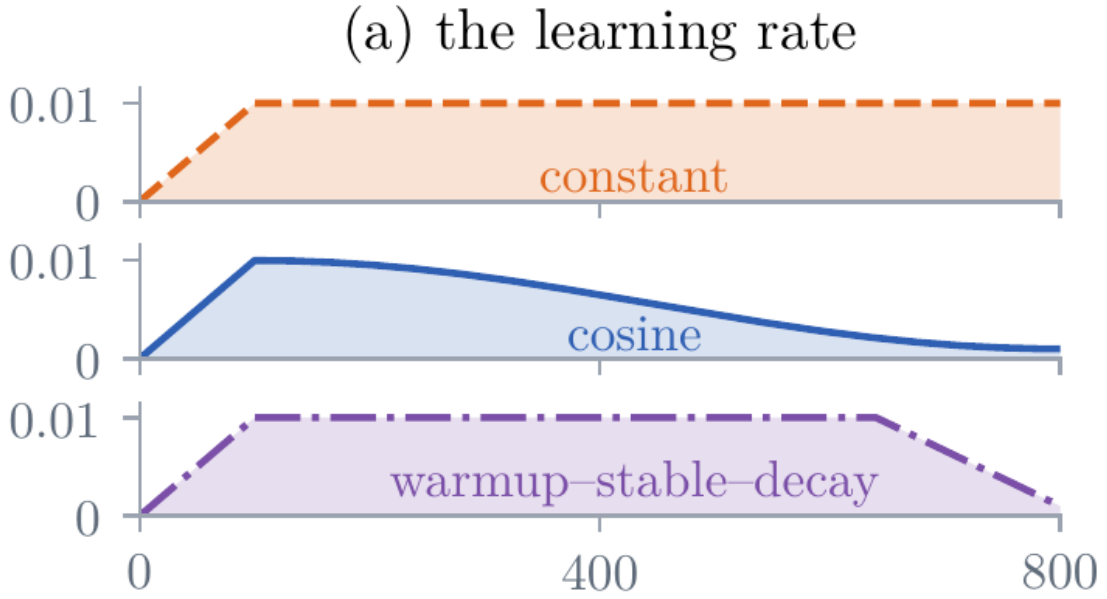


Figure 26a: warmup–stable–decay (purple); the decay at the end

Midtraining

Our midtraining run

1,000 steps of 16 chunks of 512 tokens: 8 from the pretraining stream, 8 from a stream of conversations, packed alike

46,000 conversations, 11.0 million tokens: story requests, sums, and a new task, summarizing a story in one sentence

the loss on every token: the system's and the user's included

AdamW, continued: a warmup of 50 steps to 4×10^{-4}, constant, then a linear decay to zero over the last 30%

56 minutes: on a CPU

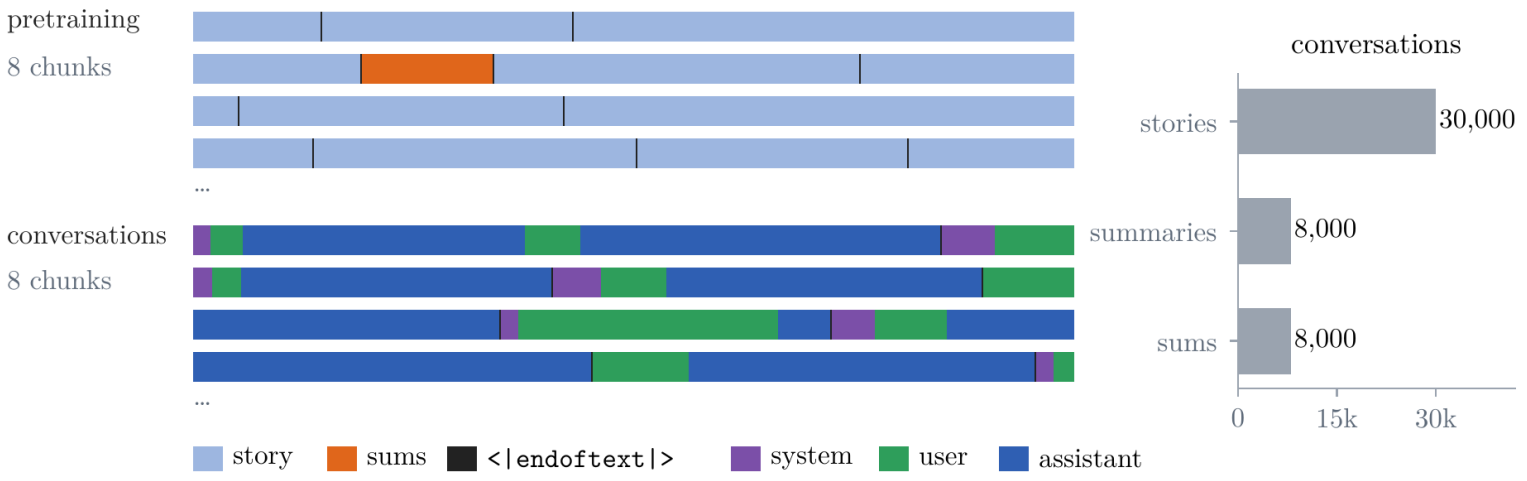


Figure 42: one step's 16 chunks; right, the conversations by kind

Midtraining

What midtraining changed

the format: a story about Spot and a bone, then <|end|>; 85 of 100 held-out answers ended, against none

the assistant's turns: down to what fine-tuning the base model directly gives, 1.90 → 1.63 nats per token on 300 story conversations

the stories kept: 1.818, up to 1.858 by step 100, then 1.813 after the decay

a new task: summaries of unseen stories; ROUGE-1, the F1 of shared words, 0.27 against 0.15 for the first sentence

not the sums: 7% of unseen sums right; a sixth of the conversations, but 2.7% of their tokens, and a mixture is weighted by tokens

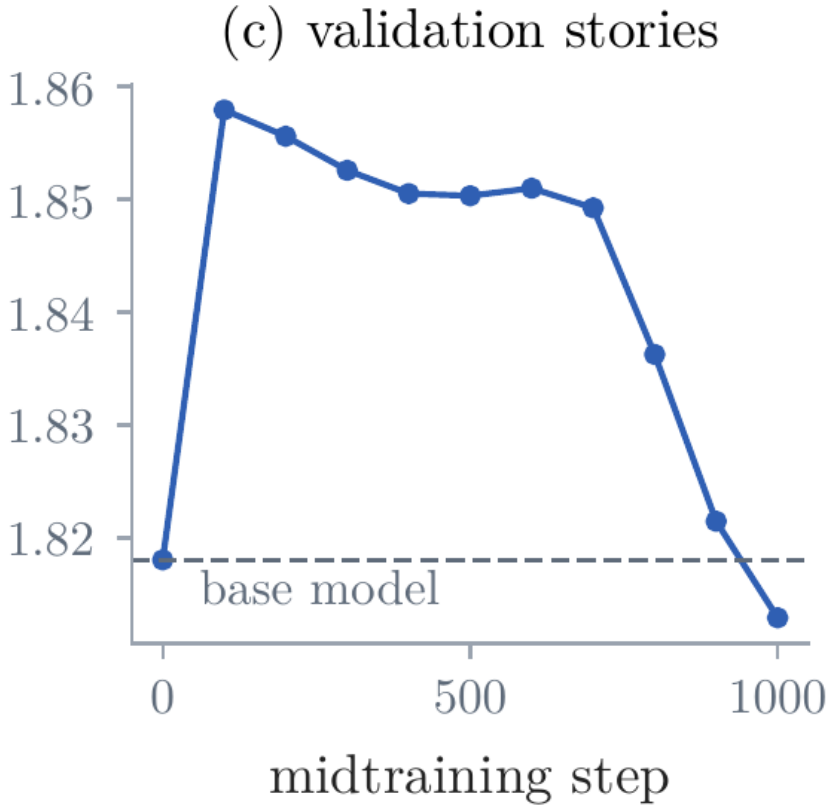


Figure 43c: the 500 validation stories

Midtraining

A new task: summarizing

a held-out story: the midtrained model's summary, and the dataset's

ROUGE-1: precision and recall of shared words, and their harmonic mean (Lin, 2004)

on 100 held-out stories: 0.27, against 0.15 for the first sentence and 0.08 for the base model

(b) summarizing, the new task: a held-out story

user

Summarize this story in one sentence.
Once upon a time, there was a little girl named Lily. She loved watching TV, especially her favorite show about animals. One day, her mom told her it was time to turn off the TV and go outside to play. Lily didn't want to stop watching her show, so she got upset and started to cry.
Her mom tried to calm her down and suggested they go outside and play together. They went to the park and saw an amazing butterfly. Lily wanted to catch it, but she didn't know how. Her mom showed her how to fold her hands together to make a butterfly net. They caught the butterfly and watched it for a while before letting it go.
Lily realized that there were other amazing things to do besides watching TV. She was happy to spend time with her mom and learn new things. From that day on, she didn't mind turning off the TV to go outside and explore the world.

assistant: the midtrained model

Lily loves to watch TV and learns about the butterfly in the park, but her mom tells her it's time to turn off the TV and learn to be patient. `<|end|>`

TinyStories-Instruct's summary

Lily loves watching TV but her mom takes her outside to play and they catch a butterfly, making Lily realize there are other amazing things to do besides watching TV.

Figure 44b: a held-out story, the midtrained model's summary, and TinyStories-Instruct's

Midtraining

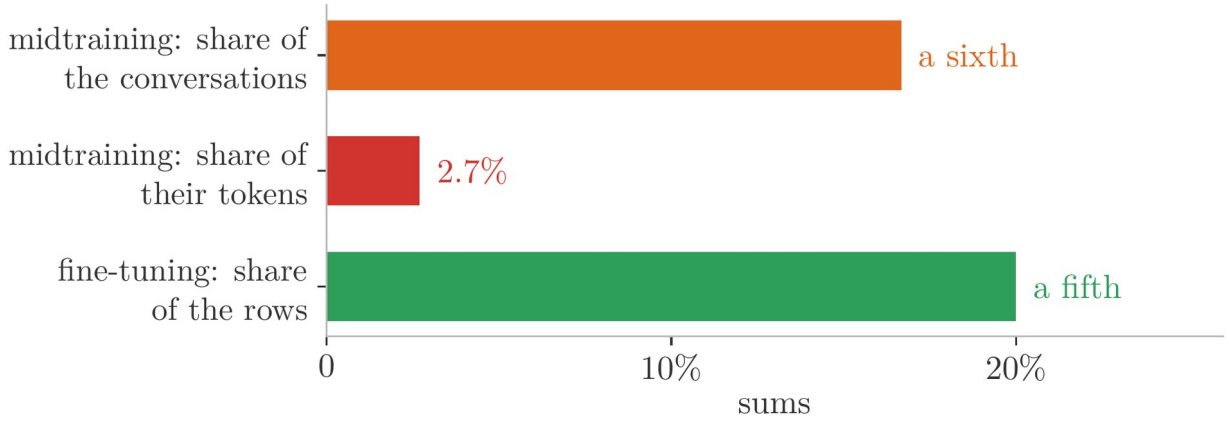
What a mixture's weights mean

the sums are not learned: for *237 + 682* it writes *137 + 282 = 429.*; 7% of unseen sums right

a sixth of the conversations: but, being short, 2.7% of their tokens; packed chunks weigh a conversation by its tokens

in 1,000 steps: about 3,000 conversations of sums; fine-tuning gives them a fifth of its rows

so: a mixture is set in tokens; a skill shown in few tokens needs more weight than its share of examples



Slide figure: the sums' share in midtraining and in fine-tuning

Supervised Fine-Tuning

The loss on the assistant's tokens

the problem: midtraining reads conversations as text, cut anywhere, every token predicted → **the fix here**: whole conversations, and the loss on the assistant's tokens only

$$\mathcal{L}_{\mathrm{SFT}}(\theta) = -\frac{1}{|A|}\sum_{t\in A}\log q_\theta(x_t \mid x_{<t})$$

only the assistant's tokens: A, the positions whose target is in an assistant turn; <|end|> is in A, the role tokens are not

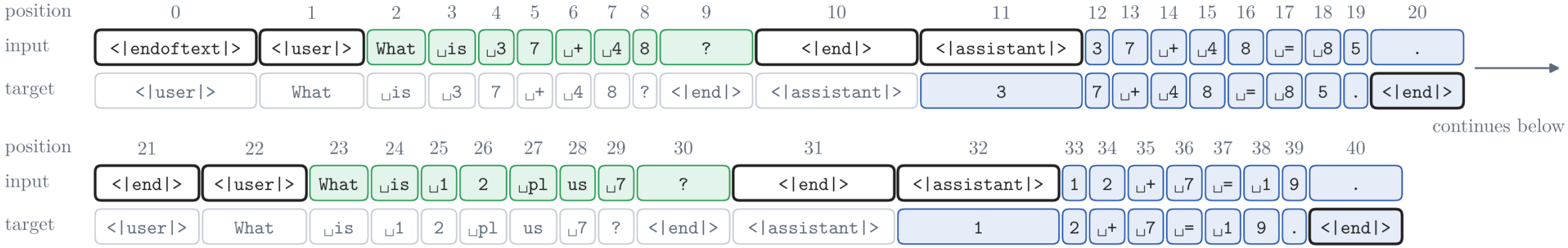


Figure 45, in two rows: inputs (top) and targets (bottom); in the loss, blue

every position reads all before it: the mask leaves targets out of the loss, not inputs out of attention

Supervised Fine-Tuning

Demonstrations

written by people: InstructGPT, answers to about 13,000 prompts; ChatGPT, conversations with people on both sides

generated by a model: from a few examples written by people, then filtered (Self-Instruct)

distilled from a larger model: Alpaca, LLaMA 7B on 52,000 answers of text-davinci-003

ours, from TinyStories-Instruct: a storyteller's system message, a sentence per feature; a request; the story

24,179 conversations, 5.3 million tokens: one in ten with a second turn; sums; no summaries

(a) a summary as the request; one feature, dialogue

system
You are a storyteller for young children. In your stories, the characters talk to each other.

user
Can you tell me a story where Lily asks her mom for grapes, but they don't have any. Her dad comes home with a bag of grapes and Lily is happy?

assistant
Once upon a time, there was a little girl named Lily. She loved to eat grapes, but she didn't have any. She asked her mom, "Mommy, I want grapes, please?"
Her mom said, "I'm sorry, Lily. We don't have any grapes right now."
Lily felt frustrated because she really wanted grapes. Suddenly, her dad came home with a big bag of grapes! Lily was so happy and said, "Yay! Grapes! Thank you, Daddy!"

Figure 46a: one of our fine-tuning conversations, a summary as the request and one feature, dialogue

Supervised Fine-Tuning

The recipe of fine-tuning

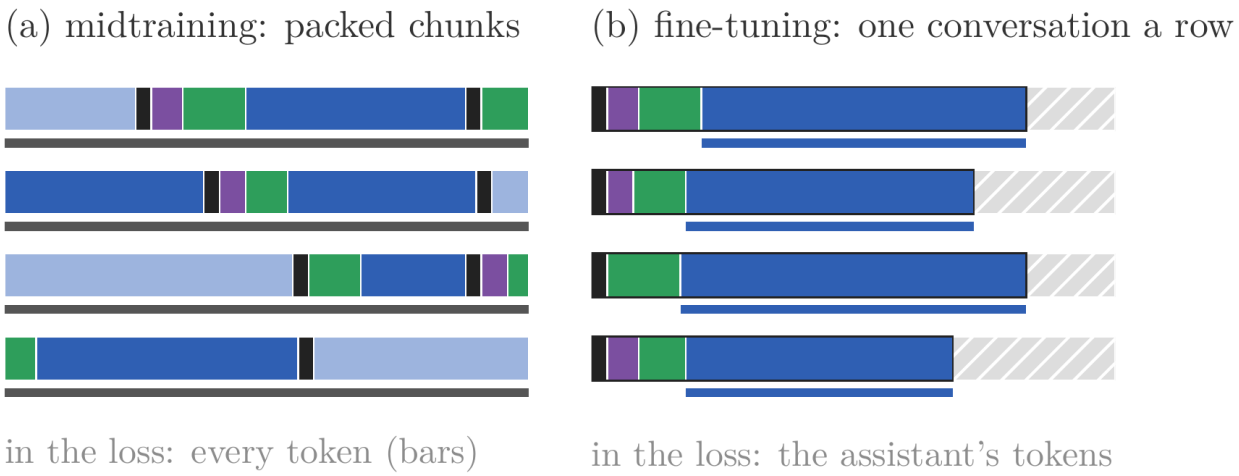
one conversation per row: whole, from <|endoftext|>, as in use; padded to the longest of 16 similar ones

the loss mask: the assistant's tokens; padding left out too

a smaller rate: a peak of 3×10^{-4}, a seventh of pretraining's; 5% warmup; cosine to a tenth

one pass: 1,512 steps; reading a few examples many times memorizes them

the optimizer: AdamW, $\beta = (0.9, 0.95)$, weight decay 0.1, clipping at 1; started afresh



Slide figure: (a) midtraining's packed chunks, (b) fine-tuning's padded rows

Supervised Fine-Tuning

Training fewer parameters: LoRA

the problem: full fine-tuning needs pretraining's 16 bytes per parameter again, for a small change → **the fix here**: train a low-rank change beside each frozen matrix

$$\mathbf{x}\,(W + AB), \qquad A \in \mathbb{R}^{d_{\text{in}} \times r},\ B \in \mathbb{R}^{r \times d_{\text{out}}}$$

LoRA (Hu et al., 2022): W frozen; A and B trained, $r(d_{in} + d_{out})$ numbers instead of $d_{in}d_{out}$

one of our attention matrices, r = 8: 3,072 numbers instead of 36,864

A random, B zero: training starts from the base model; then AB is added into W

in memory: frozen weights in bf16, 2 bytes and no state; 7B needs about 14 GB, not 112

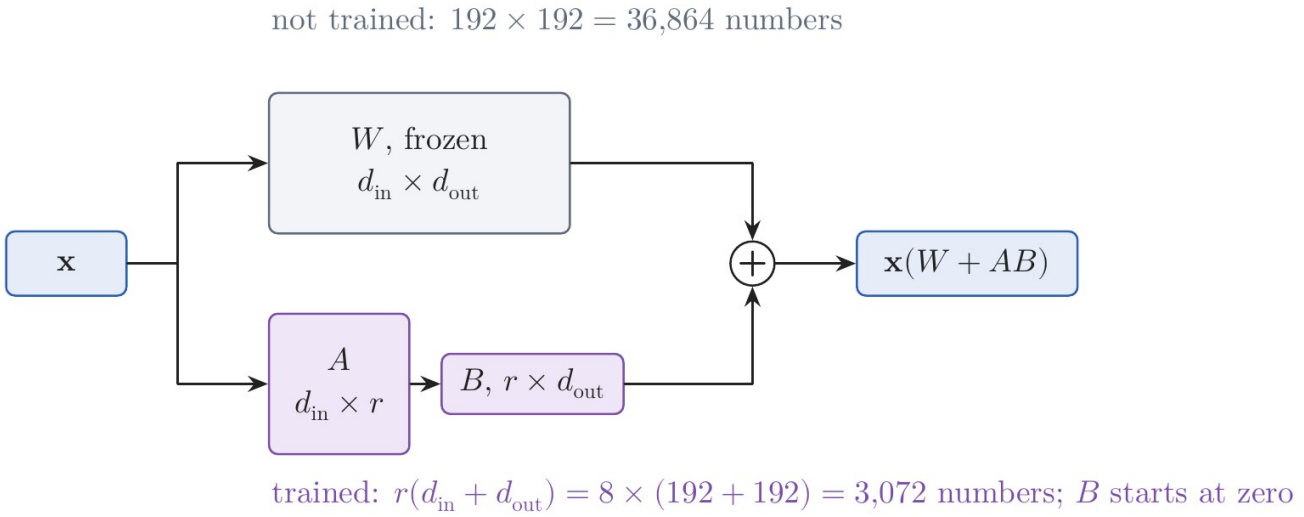


Figure 31: a frozen W (gray) and the trained path xAB (purple)

Supervised Fine-Tuning

Our chat model

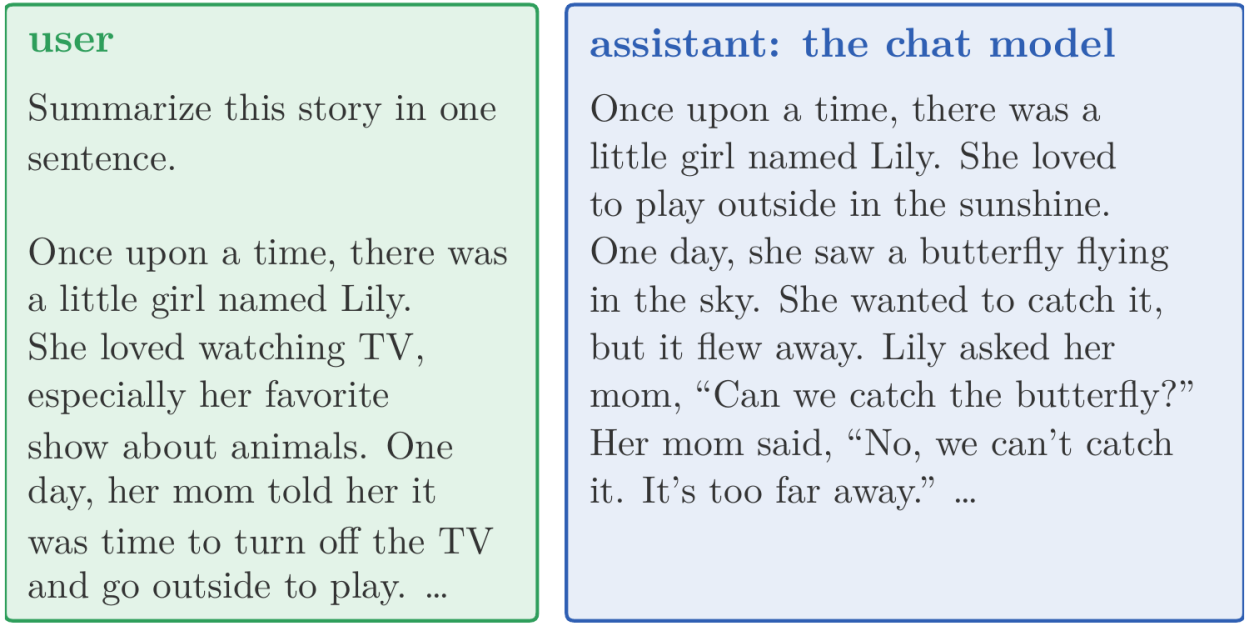
our own experiment: the chat model, midtrained and fine-tuned on all 24,179 conversations (99 of 100 answers end, 73% of sums right), against the base model fine-tuned four ways

midtraining did most of the work: fine-tuning from it starts where fine-tuning the base model ends

the mask matters little here: our prompts are a quarter of the tokens; with long prompts it matters more

LoRA must change the new tokens: with the table frozen, it almost never ends its turn; with the four rows trained, it does

a task left out is lost: fine-tuning has no summaries, and asked for one, the chat model tells a story



Slide figure: asked to summarize Figure 44b's story, the chat model tells a story (greedy, cut at 80 tokens)

Supervised Fine-Tuning

The running example, answered

base model: the conversation as plain text

You are a storyteller for young children.
Tell me a short story about a dog who finds a bone.
“Let’s share the bone!” he said, feeling surprised.
“No, I don’t think the dog’s owner, mother!”
The dog felt sad and angry. “But why?” he asked.
“You can’t have any bone,” said the dog sadly.
The owner, who was watching him, saw him trying to do better.
The end. <|endoftext|>

chat model: the conversation in its template

system: You are a storyteller for young children.
user: Tell me a short story about a dog who finds a bone.
assistant: Once upon a time, there was a dog named Max. Max was a very grateful dog. He had a shiny, red tank in his yard. Max loved to play with his tank.
One day, Max went to the store with his tank. He saw a big, brown tank in place. Max thought it was a toy, but he was excited to play with it. Max followed the tank, but it was not a toy.
Max tried to carry the tank, but it was too heavy. He was so sad that he lost his tank. The tank broke into many pieces. Max was very sad and cried a lot.
But then, a dog came to help Max find the tank. Max saw the tank … <|end|>

Figure 47: the base model continues the document; the chat model answers and stops

our chat model: a story about a dog, and the end of its turn; it follows the request loosely

on 100 held-out requests: 99 ended, 189 tokens on average, 36% of the requested words

Supervised Fine-Tuning

Following the system message

in the fine-tuning stories: 95% under “the characters talk to each other” contain dialogue, against 34%

the chat model: with or without the sentence, about the same (64% vs 62%)

within chance: 100 stories in each condition; a few points is noise

why: the requests come from the stories' summaries, and already say most of what the story will be

so: train on conversations whose system message varies on its own and decides the answer

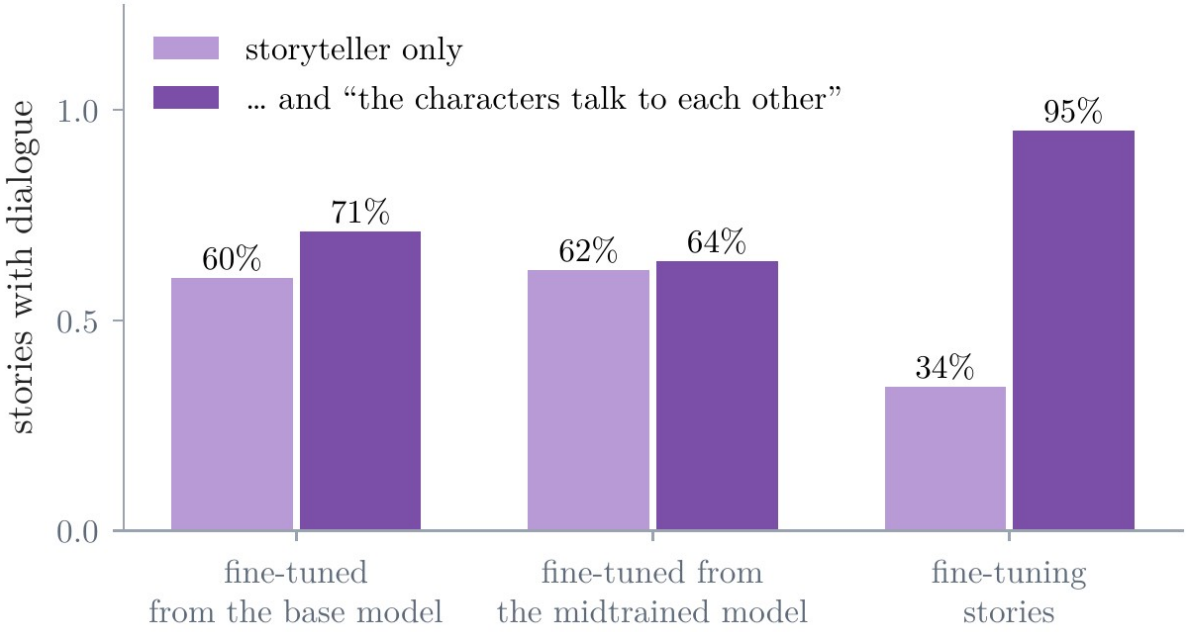


Figure 48: stories with dialogue, without (light) and with (dark) the sentence

Supervised Fine-Tuning

How many conversations?

the format, from 100: 95 of 100 answers end, about as many as 1,000 give (99)

the words need more: 10%, 21%, 35% from the base model; after midtraining, 1,000 suffice (37%)

the sums need all of them: 70% right, from the base model, only after all 24,179

LIMA (LLaMA 65B, 1,000 examples): its authors propose that fine-tuning mostly teaches the format

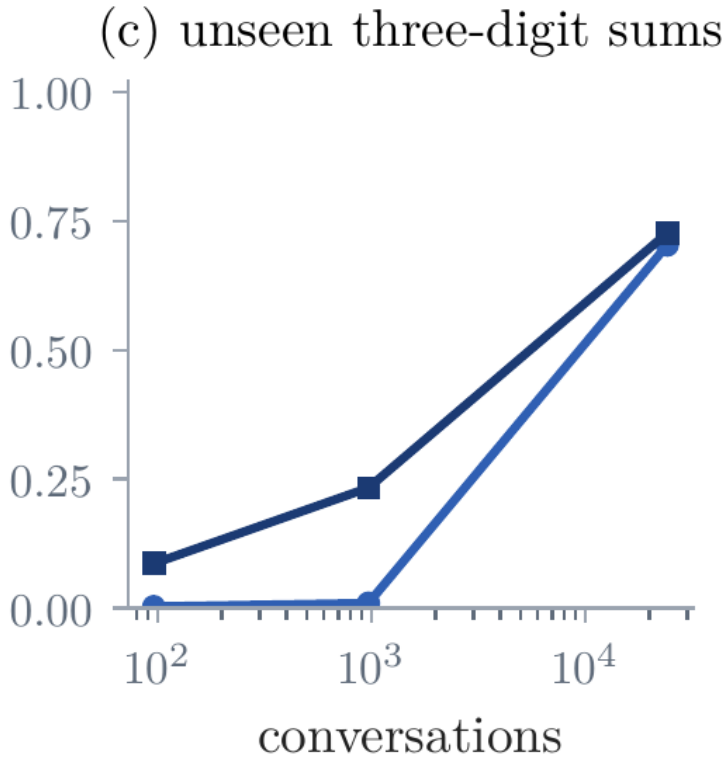


Figure 49c: unseen three-digit sums right; dark, from the midtrained model

What Fine-Tuning Changes, and What It Does Not

Every stage, built

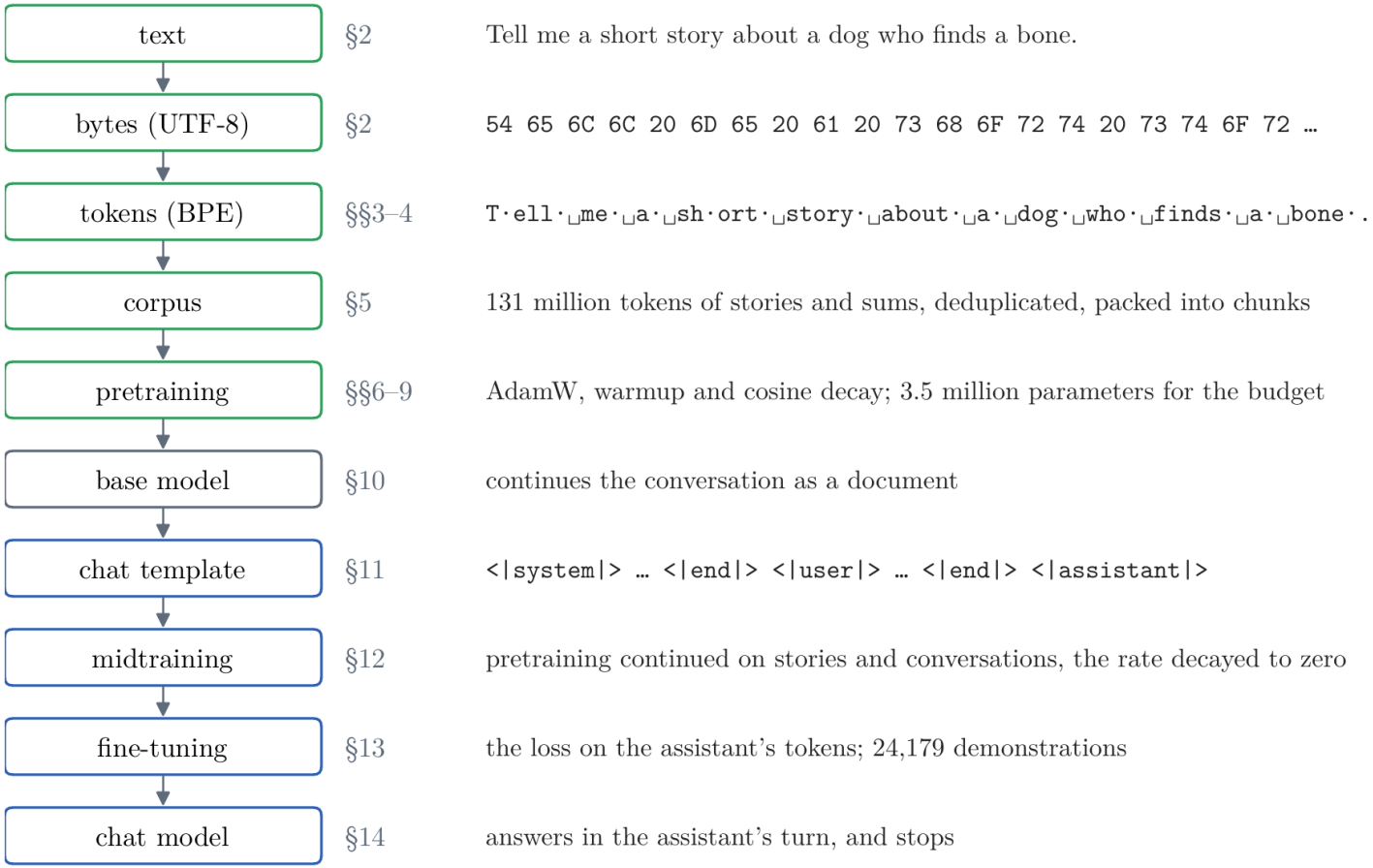


Figure 50: from text to a chat model; each stage with its section, and the running conversation as each stage sees it

ChatGPT's recipe before its rewards: in a model 50,000 times smaller than GPT-3

What Fine-Tuning Changes, and What It Does Not

What changed, and what did not

changed, the format: it answers in its turn with a story or a sum, and stops, 99 of 100

the format, not the skill: unseen sums, 73% with no examples, about the base model's 76–81% with a few, and more than its 49% without

no new knowledge: asked for the capital of France, it tells a story; new facts in fine-tuning add hallucinations (Gekhman et al., 2024)

no skill beyond the data: four-digit sums, never right

imitation compounds its errors: it never practiced recovering from its own mistakes (behavior cloning)

nothing says which answer is better: only how close it is to the demonstration; that needs rewards (Weeks 6–7)

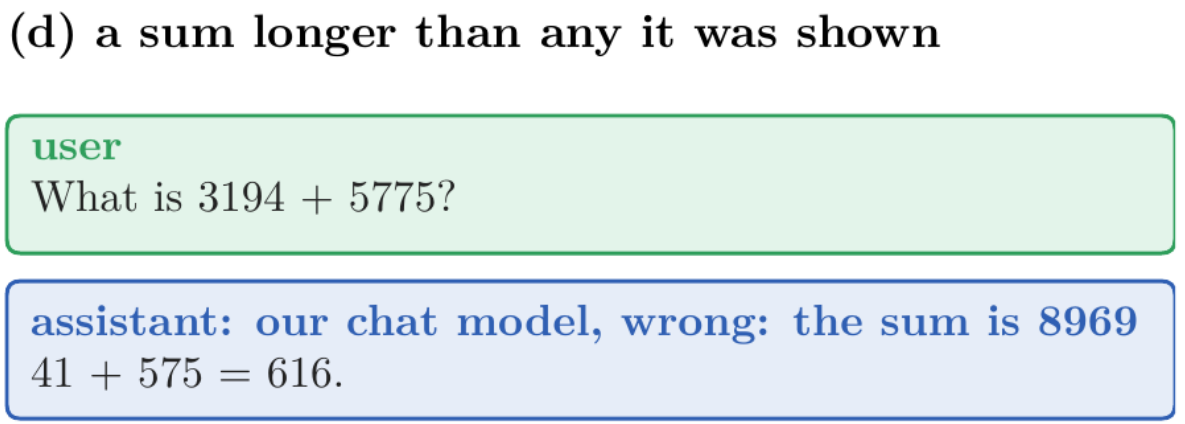


Figure 51d: a sum longer than any it was shown (greedy)

What Carries Forward

From text to a chat model, and what comes next

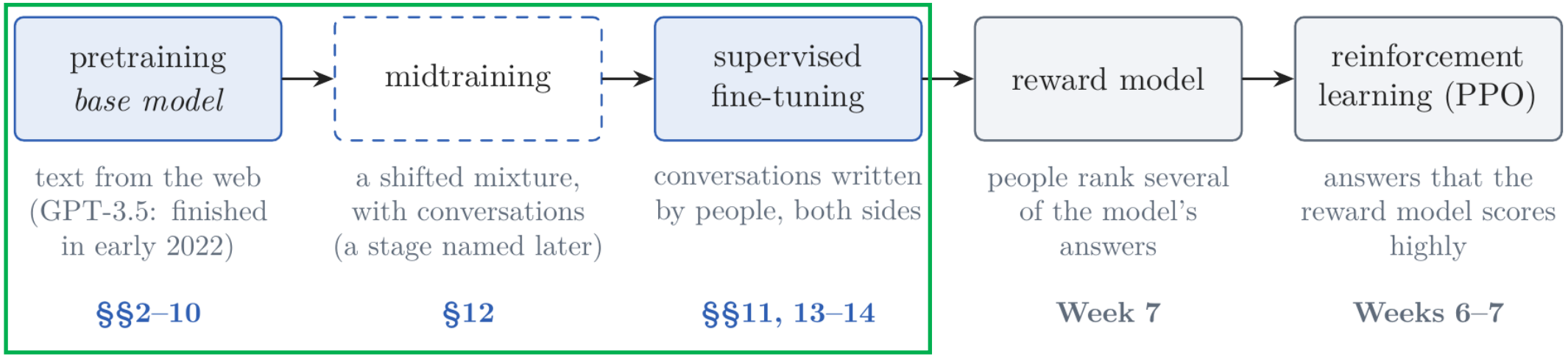


Figure 4: this lecture built the first three stages (green); Weeks 6 and 7, the rest

tokens: bytes, merged by BPE into 4,096 tokens; special tokens for structure; bits per byte

pretraining: AdamW or Muon, a batch in tokens, warmup and decay; about 20 tokens per parameter; a base model continues documents

a chat model: system, user, and assistant turns in a template; midtraining and fine-tuning on demonstrations teach the format, and add no knowledge

next week, reinforcement learning: rewards for what the model does; Week 7, ChatGPT's last two steps